Playmaker: computer defence

Code at commit c7da7b9 2026-11-05. Built 2026-11-05. Line numbers are the real ones in each file.

Everything the computer uses to choose the **defence's** actions, including the setup placement.

The path of a decision. `Computer.choose` → `_defend`: habits that override first (jumps the first pass, nearest closes out), the contest choice (`_contest`), then the generator's defence or reaction plays, filtered by the level's concepts (`_concept_filter`), each valued by the offence's best answer (`_value_fn` → `_threat`, which also weighs a dribble), and picked with the comfort line (`_comfort_key`) and the habits (`_habit_rewrite`). The setup is `place_defence`.

Contents

Section	Lines	What it is
computer.py (shared core)	517	The Computer framework, card reading, reachable contests.
solver.py (shared core)	730	The evaluation both sides use, defender reach, applying plays.
solver_config.py	49	Evaluation widths and weights.
computer_levels.json (shared keys)	48	Thinking time, level names.
computer.py (defence)	234	Defence choice, comfort line, habits, contests, setup.
defence.py	161	Arrow jobs; the step-3e reading defence.
solver.py (defence plays)	210	Defence and reaction candidate plays.
computer_levels.json (defence)	84	The defence presets: every lever.
policies.py	156	The base class; the greedy baseline.
solver.py (search, appendix)	260	The search; not used by any level now.

Part 1. The shared core (the same in both computer documents)

The computer player framework, card reading, contest reach, and the solver's evaluation and reach model that both sides use to value positions.

playmaker_engine/computer.py

Shared: the Computer class framework (choose, queue, values, ranking), card reading by level, reachable contests.

```
"""The computer opponent, both sides, at three levels (step 3e).

The computer chooses among legal actions only; no rule changes. It sees what a human player sees: the
cards (read exactly, by their marks, or by position, by level), the board and the trackers. It never
reads the Coach's Call target when defending, gets no extra tokens, and uses no stored solver results.

How it values things:
- A contest is valued after the defence's legal reaction (`reachable_contest`): a defender with their
  Move unused, not locked and a token in the pool can still get adjacent and facing once the shot is
  declared; Quick Step keeps the facing, a Sprint faces the way it ran.
- A position is valued by the offence's best next action against it (shoot now, drive, or pass and
  then shoot), the solver's evaluation. A level may instead use the solver's search at a short budget
  (lookahead 'solver'); calibration found the one-turn ranking stronger within the time limit.
- Concepts, habits, tunnel vision, card reading, the comfort line and randomness are the level levers
  (computer_levels.json).
"""
import gc
import json
import os
import random
import time

from . import board as B
from . import rules_config as RC
from .actions import Action, legal_actions, MOVEMENT
from .cards import CARDS
from .policies import Policy, expected
from .resolve import apply, ChanceNode
from .state import Context

with open(os.path.join(os.path.dirname(__file__), 'computer_levels.json')) as _f:
    LEVELS = json.load(_f)
LEVEL_NAMES = ('rookie', 'starter', 'allstar')
BUDGET = dict(LEVELS['budget_s'])          # 'server' / 'browser'; the browser build lowers 'active'
BUDGET['active'] = BUDGET['server']


# ---- the reaction (section 1) ----------------------------------------------------------

def reaction_budget(state):
    """Actions the defence may spend reacting to a declared shot: 0, 1 or 2 (OQ-6), from its tokens."""
    return min(2, state.tok_x) if state.tok_x > 0 else 0


def reachable_contest(state, shooter=None, shot=None, budget=None):
    """Every defender who could contest a shot by `shooter` (default: the ball carrier) from where the
    shooter stands, after the defence's legal reaction. Each entry: defender, clean (arrow on the shooter
    already, or the reaction gets them adjacent and facing), actions (reaction actions needed), contest
    (rating after the -3 when further from the rim than the shooter), value (shot points after this
    contest; a foul roll entry also gives the foul probability). The reaction budget is shared: the
```

```
    contest is made by one defender, who may use the whole budget."""
    from .solver import defender_reach
    from .scoring import foul_on_shot_points
    shooter = state.ball if shooter is None else shooter
    spos = state.o[shooter]
    z = B.zone(spos)
    types = RC.ZONE_SHOTS[z]
    if not types:
        return []
    t = shot or max(types, key=lambda k: state.ctx.off[shooter].shot(k))
    pts = RC.ZONE_POINTS[z]
    card = state.ctx.off[shooter]
    budget = reaction_budget(state) if budget is None else budget
    rules = state.ctx.rules
    if rules.contest_geometry != 'current' or rules.arrow_mode != 'single':
        return _reachable_contest_geo(state, shooter, spos, t, pts, card, budget)
    out = []
    for d in range(5):
        free, fixed = defender_reach(state, d, budget)
        best = None
        for q in set(free) | set(fixed):
            if not B.adjacent(q, spos) or q in state.o:
                continue
            on = B.dir_between(q, spos)
            far = B.dist_rim(q) > B.dist_rim(spos)
            c = state.ctx.dfn[d].con(t) - (RC.FAR_CONTEST_PENALTY if far else 0)
            if state.ctx.rules.floor_contest_zero:
                c = max(0, c)
            clean_len = len(free[q]) if q in free else None
            for a, seq in fixed.get(q, {}).items():
                if a == on and (clean_len is None or len(seq) < clean_len):
                    clean_len = len(seq)
            if clean_len is not None:
                cand = dict(defender=d, square=B.sq(q), clean=True, actions=clean_len, contest=c,
                            value=max(0, pts * (card.shot(t) - c)))
            else:
                ln = min(len(seq) for seq in fixed[q].values())
                pf = state.ctx.dfn[d].r['foul_contest'] / 20.0
                fp = float(foul_on_shot_points(card, t, pts, state.ctx.rules)[0])
                v = max(0, pts * (card.shot(t) - c))
                cand = dict(defender=d, square=B.sq(q), clean=False, actions=ln, contest=c, foul_p=pf,
                            value=pf * fp + (1 - pf) * v)
            if best is None or (cand['value'], cand['actions']) < (best['value'], best['actions']):
                best = cand
        if best:
            out.append(best)
    return sorted(out, key=lambda e: e['value'])


def _reachable_contest_geo(state, shooter, spos, t, pts, card, budget):
    """reachable_contest under a step 3h setting (a wedge, or a positional contest geometry), through
    geometry.best_terms: each entry also names its category (front / side / behind)."""
    from . import geometry as G
    from .solver import defender_reach
    from .scoring import foul_on_shot_points
    rules = state.ctx.rules
    out = []
    for d in range(5):
        free, fixed = defender_reach(state, d, budget)
        best = None
        for q in set(free) | set(fixed):
            if not B.adjacent(q, spos) or q in state.o:
                continue
            opts = []
            if q in free:
                opts.append((None, len(free[q])))
            opts += [((a,), len(seq)) for a, seq in fixed.get(q, {}).items()]
            for arrows, ln in opts:
                bt = G.best_terms(rules, q, arrows, spos)
                if bt is None:
                    continue
                _, foul_roll, penalty, cat, _ = bt
                c = state.ctx.dfn[d].con(t) - (RC.FAR_CONTEST_PENALTY if penalty else 0)
                if rules.floor_contest_zero:
                    c = max(0, c)
                v = max(0, pts * (card.shot(t) - c))
                if foul_roll:
                    pf = state.ctx.dfn[d].r['foul_contest'] / 20.0
                    fp = float(foul_on_shot_points(card, t, pts, rules)[0])
                    cand = dict(defender=d, square=B.sq(q), clean=False, actions=ln, contest=c, foul_p=pf,
                                value=pf * fp + (1 - pf) * v, category=cat)
                else:
                    cand = dict(defender=d, square=B.sq(q), clean=True, actions=ln, contest=c, value=v, category=cat)
                if best is None or (cand['value'], cand['actions']) < (best['value'], best['actions']):
                    best = cand
        if best:
            out.append(best)
    return sorted(out, key=lambda e: e['value'])


# ---- passing judgement (step 3f) ------------------------------------------------------------

_PASS_CACHE = {}
```

… lines 146–217: printed in the offence document …

```

```

… lines 220–233: printed in the offence document …

```python
# ---- what a side can see; card reading (section 8 lever 4) ----------------------------------------

def _positions():
    from .scouting import position
    return {k: position(c) for k, c in CARDS.items()}


_POS = None
_MIDS = None


def _mids():
    """Middle value of each band of each rating, and each position's typical band and QS, from the 75 cards."""
    global _MIDS, _POS
    if _MIDS is None:
        _POS = _positions()
        vals, by_pos = {}, {}
        for c in CARDS.values():
            for r, v in c.r.items():
                vals.setdefault((r, c.bands[r]), []).append(v)
                by_pos.setdefault((_POS[c.key], r), []).append(c.bands[r])
        mid = {k: round(sum(v) / len(v)) for k, v in vals.items()}
        typical = {k: max(set(v), key=v.count) for k, v in by_pos.items()}
        qs = {}
        for c in CARDS.values():
            qs.setdefault(_POS[c.key], []).append(c.qs)
        qs = {p: sorted(v)[len(v) // 2] for p, v in qs.items()}
        _MIDS = (mid, typical, qs)
    return _MIDS


class SeenCard:
    """A card as the computer reads it at its level: 'exact' (the numbers), 'marks' (only ▲ / plain / ▼:
    every rating is the middle of its band) or 'role' (judged by position: the position's typical band)."""
    __slots__ = ('key', 'num', 'role', 'tier', 'tagline', 'qs', 'r', 'bands')

    def __init__(self, card, reading):
        mid, typical, qs = _mids()
        self.key, self.num, self.role, self.tier, self.tagline = card.key, card.num, card.role, card.tier, card.tagline
        if reading == 'marks':
            self.bands = dict(card.bands)
            self.qs = card.qs
        else:                                   # role
            pos = _POS[card.key]
            self.bands = {r: typical[(pos, r)] for r in card.r}
            self.qs = qs[pos]
        self.r = {r: mid[(r, b)] for r, b in self.bands.items()}

    def shot(self, t):
        return self.r['shot_' + t]

    def con(self, t):
        return self.r['con_' + t]


def seen_context(ctx, reading):
    if reading == 'exact':
        return ctx
    return Context([SeenCard(c, reading) for c in ctx.off], [SeenCard(c, reading) for c in ctx.dfn], ctx.rules,
                   ctx.title, ctx.kind, ctx.scenario, ctx.card_clock, ctx.inbounder, ctx.coach_call,
                   ctx.three_only, ctx.two_or_three, ctx.hold_for_one, ctx.off_team, ctx.def_team)


VISIBLE_FIELDS = ('o', 'x', 'arrows', 'ball', 'clock', 'phase', 'side', 'stage', 'tok_o', 'tok_x', 'pending',
                  'shot', 'screens', 'locks', 'moved', 'counts')


def seen_state(state, side, ctx):
    """The state as `side` may see it: the defence never sees the offence's secret Coach's Call target."""
    s = state.with_(ctx=ctx)
    if side == 'X' and state.ctx.coach_call and state.stage != 'over':
        s = s.with_(target=-1)
    return s


# ---- the level evaluation: the solver's evaluation, with the concepts as switches -----------------

def _evaluator_class():
    from .solver import Evaluator

    class LevelEvaluator(Evaluator):
        """reaction: count reaction-reachable contests (off: only the arrows as they stand);
        reserve: see that a spent token or Move loses the reaction (off: always assume a reaction);
        tunnel: only the N attackers nearest the ball are a threat."""

        def __init__(self, cfg, reaction=True, reserve=True, tunnel=5):
            super().__init__(cfg)
            self.reaction, self.reserve, self.tunnel = reaction, reserve, tunnel

        def reach(self, s, budget):
            if not self.reaction:
                budget = 0
            elif not self.reserve:
                s = s.with_(moved_x=0, tok_x=max(s.tok_x, 2))
                budget = 2
            return super().reach(s, budget)

```

```
        def visible(self, s):
            if self.tunnel >= 5:
                return None
            cp = s.o[s.ball]
            order = sorted(range(5), key=lambda i: (i != s.ball, B.cheb(s.o[i], cp), i))
            return set(order[:self.tunnel])

        def carrier_look(self, s, shooter, budget_x, can_move, forced=False):
            vis = self.visible(s)
            if vis is not None and shooter not in vis:
                return (0.0, 0.0, 0.0, None, None)
            return super().carrier_look(s, shooter, budget_x, can_move, forced)

    return LevelEvaluator


# ---- the computer player ---------------------------------------------------------------

class Computer(Policy):
    """One side's computer at a level. choose(state) -> a legal action. notes: decisions to log (plans,
    targets, habits that fired)."""
    name = 'computer'

    def __init__(self, side, level='starter', seed=0, overrides=None, habits=None, budget_s='active'):
        if level not in LEVEL_NAMES:
            raise ValueError(f'level must be one of {LEVEL_NAMES}')
        self.side, self.level = side, level
        part = LEVELS['defence' if side == 'X' else 'offence']
        self.cfg = json.loads(json.dumps(part['levels'][level]))
        for k, v in (overrides or {}).items():
            if isinstance(v, dict) and isinstance(self.cfg.get(k), dict):
                self.cfg[k].update(v)
            else:
                self.cfg[k] = v
        self.rng = random.Random(seed * 7919 + (1 if side == 'O' else 2))
        pool = [h for h in part['habits'] if h not in self.cfg.get('habits_not', [])]
        self.habits = list(habits) if habits is not None else self.rng.sample(pool, self.cfg['habits'])
        self.budget_s = BUDGET['active'] if budget_s == 'active' else budget_s
        self.comfort = (LEVELS['defence']['comfort_line'] + self.cfg.get('comfort_offset', 0)) if side == 'X' else None
        self.notes = []
        self.stats = dict(decisions=0, comfort_decided=0, habit_fired={}, seconds=[])
        self._ctx = None
        self.plan = None
        self.queue = None

    # -- setup per possession (the solver's caches key positions without their context) --
    def _context(self, state):
        if state.ctx is self._ctx:
            return
        from .solver import Generator, Searcher
        from .solver_config import with_config
        self._ctx = state.ctx
        self.seen = seen_context(state.ctx, self.cfg['reading'])
        c = self.cfg['concepts']
        reaction = c.get('reaction', c.get('reaction_shot', True))
        reserve = c.get('reserve', True) if self.side == 'X' else c.get('drain', True)
        # no root-wide raw action scan and no exact endgame hand-off: neither watches the clock
        self.ecfg = with_config(eval_turns=1, top_k_raw=0, endgame_tokens=-1, eval_reply_k=2)
        self.ev = _evaluator_class()(self.ecfg, reaction=reaction, reserve=reserve, tunnel=self.cfg['tunnel'])
        self.gen = Generator(self.ecfg, self.ev)
        self.ev._gen = self.gen
        self.Searcher = Searcher
        self.queue = None
        self.plan = None
        self.visited = {}           # offence: (squares, ball) reached at its decisions this possession -> times
        self.aplan = None           # offence: the advantage play being worked (step 3g)
        self._handover = None       # offence: the board as it left it to the defence (for help_pulled)
        self.picked_off = set()     # offence: passes intercepted this possession, keyed by the full position
        self._last_pass = None

    def describe(self):
        """For the log and the review: level, habits drawn, concepts switched off."""
        off = [k for k, v in self.cfg['concepts'].items() if not v and k != 'reaction_shot']
        return dict(level=self.level, name=LEVELS['names'][self.level][0], habits=self.habits, concepts_off=off)

    def _fire(self, habit):
        self.stats['habit_fired'][habit] = self.stats['habit_fired'].get(habit, 0) + 1

    # -- choose --
    def choose(self, state):
        """A legal action. Python's cycle collector is paused while it thinks: the search's caches hold
        millions of small positions, and a full collection walking them cost up to 5 seconds a decision.
        Positions form no reference cycles, so reference counting still frees them as it goes."""
        was = gc.isenabled()
        gc.disable()
        try:
            return self._choose(state)
        finally:
            if was:
                gc.enable()

    def _choose(self, state):
        acts = legal_actions(state)
        if len(acts) == 1:
            return acts[0]
        self._context(state)
        if self.side == 'O' and state.stage == 'play' and state.side == 'O':
```

```
            self._remember(state)
        t0 = time.time()
        self._deadline = t0 + self.budget_s if self.budget_s else None
        if self.queue and self.queue[0] == state and self.queue[1] and self.queue[1][0] in acts:
            a = self.queue[1][0]
            self._queue(state, a, self.queue[1][1:])
            self._note_pass(state, a)
            self._note_handover(state, a)
            return a
        self.queue = None
        view = seen_state(state, self.side, self.seen)
        if self.side == 'X':
            play = self._defend(view, acts)
        else:
            play = self._attack(view, acts)
        a = play.actions[0] if play is not None else acts[0]
        if a not in acts:
            a = acts[0]
        self._queue(state, a, play.actions[1:] if play is not None else ())
        self._note_pass(state, a)
        self._note_handover(state, a)
        self.stats['decisions'] += 1
        self.stats['seconds'].append(time.time() - t0)
        return a

    # -- offence memory over the possession: no loops, no repeating a pass that was just picked off --
… lines 456–458: printed in the offence document …

… lines 460–465: printed in the offence document …

… lines 467–469: printed in the offence document …

… lines 471–479: printed in the offence document …
    def _queue(self, state, a, rest):
        nxt = apply(state, a)
        self.queue = (nxt, tuple(rest)) if rest and not isinstance(nxt, ChanceNode) else None

    # -- values --
    def _immediate(self, s):
        """Lookahead 0: the ball carrier's shot as it stands (after the reaction, per the concepts)."""
        if s.stage != 'play' or s.pending:
            return self.ev(s)[0].ev
        base = RC.STEAL_FOUL_BONUS * s.bonus5 + RC.TURNOVER_POINTS * s.tos
        ev = self.ev.carrier_look(s, s.ball, reaction_budget(s), False)[0]
        return base + ev

    def _node_value(self, node, fn):
        if isinstance(node, ChanceNode):
            if self.side == 'O' and not self.cfg['concepts'].get('risk', True):
                return max(self._node_value(sub, fn) for _, _, sub in node.outcomes)
            return sum(float(p) * self._node_value(sub, fn) for p, _, sub in node.outcomes)
        return fn(node)

    def _value_fn(self):
        la = self.cfg['lookahead']
        if la == 0:
            return self._immediate
        if self.side == 'X':
            return self._threat
        return lambda s: (self.ev._reply(s)[0].ev if s.stage == 'play' and s.side == 'X' else self.ev(s)[0].ev)
… lines 509–520: printed in the defence document …
    def _rank(self, view, plays, fn, maximise, stop):
        from .solver import apply_play
        scored = []
        for p in plays:
            if stop is not None and scored and time.time() > stop:
                break
            node = apply_play(view, p)
            if node is None:
                continue
            v = self._node_value(node, fn)
            scored.append((v, p))
        return scored

    def _pick(self, scored, maximise, key=None):
        """Best, or with the level's randomness the second best (never the main source of mistakes)."""
        if not scored:
            return None
        cost = lambda p: sum(a.kind != 'end_turn' for a in p.actions) * 1e-6
        k = key or (lambda vp: (-vp[0] if maximise else vp[0]) + cost(vp[1]))
        ranked = sorted(scored, key=k)
        if len(ranked) > 1 and self.rng.random() < self.cfg['randomness']:
            return ranked[1][1]
        return ranked[0][1]

    @staticmethod
    def _with_end(view, acts, plays):
        """Mid-turn (one action already taken), stopping is a choice too: the generator builds plays for a
        fresh turn and leaves it out."""
        from .solver import Play
        end = next((a for a in acts if a.kind == 'end_turn'), None)
        if end is not None and view.stage == 'play' and view.turn_n >= 1 and not any(p.actions == (end,) for p in plays):
            plays = list(plays) + [Play('end', (end,))]
        return plays

    def _stop(self):
```

```
        return getattr(self, '_deadline', None)

    def _late(self, share=0.85):
        """Most of the thinking time already gone (the search used it): fall back to the quick value."""
        d = self._stop()
        return d is not None and self.budget_s and time.time() > d - self.budget_s * (1 - share)

    # ---- defence ----
… lines 565–604: printed in the defence document …

… lines 606–620: printed in the defence document …

… lines 622–648: printed in the defence document …

… lines 650–671: printed in the defence document …

… lines 673–683: printed in the defence document …

… lines 685–693: printed in the defence document …
    def _search(self, view):
        """The solver's search, deepening until the thinking time is up (a hard stop: when not even depth
        1 finished, the caller falls back to the one-turn ranking with what time is left)."""
        from .solver import Timeout
        se = self.Searcher(self.ecfg)
        se.ev = self.ev
        se.gen = self.gen
        budget = self.budget_s or 0.5
        se.deadline = time.time() + budget * 0.5
        best = None
        try:
            for d in range(1, self.ecfg.max_depth + 1):
                se.root_depth = d
                se.value(view, d)
                e = se.tt.get((view, d))
                if e is not None and e.best is not None:
                    best = e.best
        except Timeout:
            pass
        return best

    # ---- offence ----
… lines 717–745: printed in the offence document …

… lines 747–768: printed in the offence document …

… lines 770–782: printed in the offence document …

… lines 784–859: printed in the offence document …
    # -- passing judgement (step 3f) --
… lines 862–863: printed in the offence document …

… lines 865–889: printed in the offence document …

… lines 891–903: printed in the offence document …

… lines 905–907: printed in the offence document …

… lines 909–915: printed in the offence document …

… lines 917–933: printed in the offence document …

… lines 935–956: printed in the offence document …

… lines 958–992: printed in the offence document …
    # ---- step 3g: create an advantage, keep it, finish it ------------------------------------------
… lines 995–995: printed in the offence document …

… lines 997–998: printed in the offence document …

… lines 1000–1002: printed in the offence document …

… lines 1004–1012: printed in the offence document …

… lines 1014–1018: printed in the offence document …

… lines 1020–1045: printed in the offence document …

… lines 1047–1051: printed in the offence document …

… lines 1053–1085: printed in the offence document …

… lines 1087–1154: printed in the offence document …

… lines 1156–1222: printed in the offence document …
    # -- the plan: the advantage play being worked, kept until finished or taken away --
… lines 1225–1246: printed in the offence document …

… lines 1248–1288: printed in the offence document …

… lines 1290–1297: printed in the offence document …
```

```
1298
    … lines 1299–1301: printed in the offence document …
1302
    … lines 1303–1304: printed in the offence document …
1305
    … lines 1306–1314: printed in the offence document …
1315
    … lines 1316–1321: printed in the offence document …
1322
    … lines 1323–1334: printed in the offence document …
1335 def make(side, level, seed=0, **kw):
1336     return Computer(side, level, seed, **kw)
1337
1338
1339 # ---- setup and placement (section 5) ------------------------------------------------------------
    … lines 1343–1347: printed in the defence document …
1348
    … lines 1350–1431: printed in the defence document …
1432
```

playmaker_engine/solver.py

Shared: position values (Evaluator), defender reach, plays and how they apply.

```
  1 """Step 2: the scenario solver.
  2
  3 Three parts, all on top of the unchanged engine:
  4   * exact_solve   : expectimax over raw engine actions with a transposition table (exact).
  5   * evaluate      : fast position value: the best look the offence can get on its next turn
  6                     against the defence's best two-action reaction (the regression ceiling, generalised).
  7   * Searcher      : depth-limited expectimax over basketball plays (candidate generator) with the
  8                     evaluation at the leaves, iterative deepening under a time budget, exact
  9                     probabilities at chance nodes, and the exact solver for the tail.
 10 Values are per Coach's Call target (a tuple); ordinary cards have one entry.
 11 """
 12 import random
 13 import time
 14 from dataclasses import dataclass, field
 15
 16 from . import board as B
 17 from . import geometry as G
 18 from . import rules_config as RC
 19 from .actions import Action, legal_actions, movement_dests, can_move_kind, MOVEMENT, intercept_status, t_square
 20 from .resolve import apply, ChanceNode
 21 from .scoring import score, contest_rating, foul_on_shot_points, p_success, band
 22 from .solver_config import DEFAULT
 23 from .state import CI
 24
 25
 26 class Timeout(Exception):
 27     pass
 28
 29
 30 # ============================================================================
 31 # values
 32 # ============================================================================
 33
 34 @dataclass(frozen=True)
 35 class Val:
 36     """Expected possession score, expected look, P(look >= 30), P(clock reaches 0),
 37     expected forced-shot points (counted only when the clock expires)."""
 38     ev: float
 39     look: float = 0.0
 40     p30: float = 0.0
 41     pexp: float = 0.0
 42     fs: float = 0.0
 43
 44
 45 def mix(pairs):
 46     """Weighted average of Val tuples: pairs = [(p, (Val, ...)), ...]."""
 47     n = len(pairs[0][1])
 48     out = []
 49     for t in range(n):
 50         ev = look = p30 = pexp = fs = 0.0
 51         for p, vs in pairs:
 52             v = vs[t]
 53             ev += p * v.ev
 54             look += p * v.look
 55             p30 += p * v.p30
 56             pexp += p * v.pexp
 57             fs += p * v.fs
 58         out.append(Val(ev, look, p30, pexp, fs))
 59     return tuple(out)
 60
 61
 62 def terminal_vals(s, targets):
 63     out = []
 64     for t in targets:
 65         st = s if t is None else s.with_(target=t)
 66         sc = score(st)
 67         r = s.result
 68         shot = r is not None and r.kind in ('shot', 'forced', 'free_throws')
 69         look = float(sc.look) if shot else 0.0
 70         expired = s.clock == 0
 71         out.append(Val(float(sc.total), look, 1.0 if (shot and sc.look >= RC.SCENARIO_GOAL and not s.zeroed) else 0.0,
```

```
                      1.0 if expired else 0.0, look if expired else 0.0))
    return tuple(out)


# ============================================================================
# fast evaluation
# ============================================================================

def _with_x(s, d, q, arrow=None):
    x = list(s.x)
    x[d] = q
    kw = dict(x=tuple(x))
    if arrow is not None:
        arr = list(s.arrows)
        arr[d] = arrow
        kw['arrows'] = tuple(arr)
    return s.with_(**kw)


from functools import lru_cache


@lru_cache(maxsize=None)
def _paths(kind, frm, qs, rules):
    """On-board movement paths from frm: tuple of (run direction or None, path, path as frozenset, dest)."""
    if kind == 'move':
        raw = [(None, p) for p in B.move_paths(frm, rules)]
    elif kind == 'qs':
        raw = [(None, p) for p in B.qs_paths(frm, qs, rules)]
    else:
        raw = list(B.sprint_paths(frm, qs, rules))
    return tuple((dr, p, frozenset(p), p[-1]) for dr, p in raw if all(B.on_board(q) for q in p))


def defender_reach(s, d, budget):
    """Where defender d can be after up to `budget` (<= 2) actions, and with what arrow.
    Returns (free, fixed): free[sq] = action list arriving with any arrow (a Move was used);
    fixed[sq] = {arrow: action list} for arrivals whose arrow is set (Quick Step / Sprint).
    Uses the engine's movement shapes, blocking and screen geometry directly (no state copies)."""
    rules = s.ctx.rules
    start = s.x[d]
    arrow = s.arrows[d]
    free, fixed = {}, {start: {arrow: []}}
    if budget <= 0 or s.tok_x <= 0:
        return free, fixed
    qs = s.ctx.dfn[d].qs
    occ = (set(s.o) | set(s.x)) - {start}
    scr = []
    for sc in s.screens:
        scr.append((t_square(s, sc), bool(sc[3] >> d & 1)))
    qs_left = RC.LIMITS['X']['qs'] - s.count('X', 'qs')
    once = rules.qs_once_per_player
    if once and s.quick_stepped('X', d):
        qs_left = 0                          # OQ-52: this defender's Quick Step is used (no Quick Step, no Sprint)
    if once:
        qs_left = min(qs_left, 1)            # one Quick Step per player: never two, nor a Quick Step and a Sprint
    can_move = not s.moved('X', d) and not s.locked('X', d)
    two = budget >= 2 and s.tok_x >= 2
    on_board, banned = B.on_board, B.diag_step_banned

    def ok(frm, path):
        if scr:
            prev = frm
            for q in path:
                for tsq, beat in scr:
                    if not beat and banned(prev, q, tsq, rules):
                        return False
                prev = q
            for tsq, beat in scr:
                if beat and path[-1] == tsq:
                    return False
        return True

    def dests(frm, kind):
        out = {}
        disjoint = occ.isdisjoint
        for dr, path, pset, dest in _paths(kind, frm, qs, rules):
            if dest not in out and disjoint(pset) and (not scr or ok(frm, path)):
                out[dest] = dr
        return out

    def add_fixed(q, a, seq):
        cur = fixed.setdefault(q, {})
        if a not in cur or len(seq) < len(cur[a]):
            cur[a] = seq

    def add_free(q, seq):
        if q not in free or len(seq) < len(free[q]):
            free[q] = seq

    moves = dests(start, 'move') if can_move else {}
    qss = dests(start, 'qs') if qs_left >= 1 and qs > 0 else {}
    sprints = dests(start, 'sprint') if can_move and qs_left >= 1 else {}
    if can_move:
        add_free(start, [('move', start)])
    for q in moves:
        add_free(q, [('move', q)])
```

```
    for q in qss:
        add_fixed(q, arrow, [('qs', q)])
    wedge = rules.arrow_mode == 'wedge'
    for q, dr in sprints.items():
        add_fixed(q, dr, [('sprint', q)])
        if wedge:                           # step 3h: the other edge on the anticlockwise side
            add_fixed(q, (dr - 1) % 8, [('sprint', q, (dr - 1) % 8)])
    if two:
        if qs_left >= 1 and qs > 0:
            for m in list(moves) + ([start] if can_move else []):
                for q in dests(m, 'qs'):
                    add_free(q, [('move', m), ('qs', q)])
        for q1 in qss:
            if can_move:
                for q in dests(q1, 'move'):
                    add_free(q, [('qs', q1), ('move', q)])
            if qs_left >= 2:
                for q in dests(q1, 'qs'):
                    add_fixed(q, arrow, [('qs', q1), ('qs', q)])
                if can_move:
                    for q, dr in dests(q1, 'sprint').items():
                        add_fixed(q, dr, [('qs', q1), ('sprint', q)])
                        if wedge:
                            add_fixed(q, (dr - 1) % 8, [('qs', q1), ('sprint', q, (dr - 1) % 8)])
        if qs_left >= 2 and qs > 0:
            for q1, dr1 in sprints.items():
                for q in dests(q1, 'qs'):
                    add_fixed(q, dr1, [('sprint', q1), ('qs', q)])
                    if wedge:
                        add_fixed(q, (dr1 - 1) % 8, [('sprint', q1, (dr1 - 1) % 8), ('qs', q)])
    return free, fixed


_DR = {(x, y): B.dist_rim((x, y)) for x in range(-1, B.W + 1) for y in range(0, B.H + 2)}


class ReachMaps(list):
    """Per-defender (free, fixed) maps, plus an index: square -> [(d, free_len, [(arrow, len)])]."""

    def __init__(self, maps):
        super().__init__(maps)
        idx = {}
        for d, (free, fixed) in enumerate(maps):
            squares = set(free) | set(fixed)
            for q in squares:
                fl = len(free[q]) if q in free else None
                fx = [(a, len(seq)) for a, seq in fixed.get(q, {}).items()]
                idx.setdefault(q, []).append((d, fl, fx))
        self.index = idx


class Evaluator:
    """Fast position values, cached per position."""

    def __init__(self, cfg=DEFAULT, targets=(None,)):
        self.cfg = cfg
        self.targets = targets
        self.reach_cache = {}
        self.cache = {}
        self.calls = 0

    # -- reach of every defender, cached on what it depends on
    def reach(self, s, budget):
        key = (s.o, s.x, s.arrows, s.moved_x, s.cnt[6], s.tok_x, s.lock_x, s.screens, budget)
        r = self.reach_cache.get(key)
        if r is None:
            r = ReachMaps([defender_reach(s, d, budget) for d in range(5)])
            if len(self.reach_cache) > 15000:
                self.reach_cache.clear()
            self.reach_cache[key] = r
        return r

    def defended(self, s, maps, shooter, spos, t, budget, forced=False):
        """Defence's best answer to a shot by `shooter` from spos (shot type t).
        Returns (ev, look, p30, kind) where kind names the answer ('open', 'clean', 'side', 'steal')."""
        if s.ctx.rules.contest_geometry != 'current':
            return self._defended_geo(s, maps, shooter, spos, t, budget, forced)
        ctx, rules = s.ctx, s.ctx.rules
        card = ctx.off[shooter]
        pts = RC.ZONE_POINTS[B.zone(spos)]
        rating = card.shot(t)
        if ctx.three_only and pts != 3:
            return 0.0, 0.0, 0.0, None
        floor = forced or rules.floor_declared_shot
        fp = float(foul_on_shot_points(card, t, pts, rules)[0])
        fp30 = fp >= 30
        v0 = pts * rating
        best = (float(v0), float(v0), 1.0 if v0 >= 30 else 0.0, 'open')
        best_ev = best[0]
        steal_ok = s.count('X', 'steal') < RC.LIMITS['X']['steal']
        tok = s.tok_x
        far_pen = RC.FAR_CONTEST_PENALTY
        floor_c = rules.floor_contest_zero
        ds = _DR[spos]
        idx = maps.index
        occ_o = s.o
        sx, sy = spos
```

```
        pk_cache = {}
        con_key = 'con_' + t
        for dx, dy in B.DIRS:
            q = (sx + dx, sy + dy)
            ents = idx.get(q)
            if not ents or q in occ_o:
                continue
            on = B.DIRS.index((-dx, -dy))
            far = _DR[q] > ds
            for d, fl, fx in ents:
                dcard = ctx.dfn[d]
                c = dcard.r[con_key] - (far_pen if far else 0)
                if floor_c and c < 0:
                    c = 0
                v = pts * (rating - c)
                if floor and v < 0:
                    v = 0
                v = float(v)
                clean_len = None
                if fl is not None and fl <= budget:
                    clean_len = fl
                for a, ln in fx:
                    if ln <= budget and a == on and (clean_len is None or ln < clean_len):
                        clean_len = ln
                if clean_len is not None:
                    if v < best_ev:
                        best, best_ev = (v, v, 1.0 if v >= 30 else 0.0, 'clean'), v
                else:
                    pf = dcard.r['foul_contest'] / 20.0
                    for a, ln in fx:
                        if ln <= budget and a != on:
                            ev = pf * fp + (1 - pf) * v
                            if ev < best_ev:
                                best, best_ev = (ev, ev, pf * fp30 + (1 - pf) * (v >= 30), 'side'), ev
                            break
                if steal_ok and clean_len is not None and clean_len + 1 <= budget and tok >= clean_len + 1:
                    pf = dcard.r['foul_steal'] / 20.0
                    pk = pk_cache.get(d)
                    if pk is None:
                        pk = pk_cache[d] = float(p_success(card.r['dribble'] - dcard.r['steal'], rules))
                    ev = pf * fp + (1 - pf) * (pk * v + (1 - pk) * RC.TURNOVER_POINTS)
                    if ev < best_ev:
                        best = (ev, pf * fp + (1 - pf) * pk * v, pf * fp30 + (1 - pf) * pk * (v >= 30), 'steal')
                        best_ev = ev
        return best

    def _defended_geo(self, s, maps, shooter, spos, t, budget, forced=False):
        """defended() under the step 3h contest geometries: only a defender who ends with the shooter on
        an arrow line contests; Front is a full contest, Side a foul roll, Behind a foul roll and the -3
        (geometry.terms decides, from the arrows each reach path can end with)."""
        ctx, rules = s.ctx, s.ctx.rules
        card = ctx.off[shooter]
        pts = RC.ZONE_POINTS[B.zone(spos)]
        rating = card.shot(t)
        if ctx.three_only and pts != 3:
            return 0.0, 0.0, 0.0, None
        floor = forced or rules.floor_declared_shot
        fp = float(foul_on_shot_points(card, t, pts, rules)[0])
        fp30 = fp >= 30
        v0 = pts * rating
        best = (float(v0), float(v0), 1.0 if v0 >= 30 else 0.0, 'open')
        best_ev = best[0]
        steal_ok = s.count('X', 'steal') < RC.LIMITS['X']['steal']
        tok = s.tok_x
        idx = maps.index
        occ_o = s.o
        sx, sy = spos
        con_key = 'con_' + t
        pk_cache = {}
        for dx, dy in B.DIRS:
            q = (sx + dx, sy + dy)
            ents = idx.get(q)
            if not ents or q in occ_o:
                continue
            for d, fl, fx in ents:
                dcard = ctx.dfn[d]
                opts = []
                if fl is not None and fl <= budget:
                    opts.append((None, fl))
                opts += [((a,), ln) for a, ln in fx if ln <= budget]
                facing_len = None
                for arrows, ln in opts:
                    bt = G.best_terms(rules, q, arrows, spos)
                    if bt is None:
                        continue                            # lines off the shooter: no contest
                    _, foul_roll, penalty, _, _ = bt
                    facing_len = ln if facing_len is None else min(facing_len, ln)
                    c = dcard.r[con_key] - (RC.FAR_CONTEST_PENALTY if penalty else 0)
                    if rules.floor_contest_zero and c < 0:
                        c = 0
                    v = pts * (rating - c)
                    if floor and v < 0:
                        v = 0
                    v = float(v)
                    if foul_roll:
                        pf = dcard.r['foul_contest'] / 20.0
                        ev = pf * fp + (1 - pf) * v
```

```
                        cand = (ev, ev, pf * fp30 + (1 - pf) * (v >= 30), 'side')
                    else:
                        cand = (v, v, 1.0 if v >= 30 else 0.0, 'clean')
                    if cand[0] < best_ev:
                        best, best_ev = cand, cand[0]
                if steal_ok and facing_len is not None and facing_len + 1 <= budget and tok >= facing_len + 1:
                    c = dcard.r[con_key]
                    v = float(max(0, pts * (rating - c)) if floor else pts * (rating - c))
                    pf = dcard.r['foul_steal'] / 20.0
                    pk = pk_cache.get(d)
                    if pk is None:
                        pk = pk_cache[d] = float(p_success(card.r['dribble'] - dcard.r['steal'], rules))
                    ev = pf * fp + (1 - pf) * (pk * v + (1 - pk) * RC.TURNOVER_POINTS)
                    if ev < best_ev:
                        best = (ev, pf * fp + (1 - pf) * pk * v, pf * fp30 + (1 - pf) * pk * (v >= 30), 'steal')
                        best_ev = ev
        return best

    def carrier_look(self, s, shooter, budget_x, can_move, forced=False):
        """Best (ev, look, p30, shot square, type) for `shooter` with the ball."""
        spos = s.o[shooter]
        maps = self.reach(s, budget_x)
        squares = [spos]
        if can_move:
            st = s.with_(ball=shooter)
            for kind in ('move', 'qs', 'sprint'):
                if can_move_kind(st, 'O', shooter, kind):
                    squares += list(movement_dests(st, 'O', shooter, kind, s.ctx.rules))
        best = (0.0, 0.0, 0.0, None, None)
        first = True
        for q in dict.fromkeys(squares):
            if not B.on_board(q):
                continue
            z = B.zone(q)
            for t in RC.ZONE_SHOTS[z]:
                if q != spos:
                    sh = s.with_(o=s.o[:shooter] + (q,) + s.o[shooter + 1:])
                else:
                    sh = s
                ev, look, p30, _ = self.defended(sh, maps, shooter, q, t, budget_x, forced)
                if first or ev > best[0]:
                    best = (ev, look, p30, q, t)
                    first = False
        return best

    def __call__(self, s):
        key = s
        v = self.cache.get(key)
        if v is not None:
            return v
        self.calls += 1
        v = self._eval(s)
        if self.cfg.eval_turns >= 2 and s.stage == 'play' and s.side == 'O' and s.tok_o >= 1 and not s.ctx.coach_call:
            v2 = self._two_turns(s, v)
            if v2 is not None and v2[0].ev > v[0].ev:
                v = v2
        v = self._terms(s, v)
        if len(self.cache) > 60000:
            self.cache.clear()
        self.cache[key] = v
        return v

    def _eval(self, s):
        if s.stage == 'over':
            return terminal_vals(s, self.targets)
        if s.stage in ('pick', 'forced'):
            return max((expected(apply(s, a), self) for a in legal_actions(s)), key=lambda vs: vs[0].ev)
        if s.stage == 'contest':
            return min((expected(apply(s, a), self) for a in legal_actions(s)), key=lambda vs: max(v.ev for v in vs))
        if s.stage == 'target':
            return max((self(apply(s, a)) for a in legal_actions(s)), key=lambda vs: vs[0].ev)
        ctx = s.ctx
        base = RC.STEAL_FOUL_BONUS * s.bonus5 + RC.TURNOVER_POINTS * s.tos
        if s.pending:
            from .actions import completed_view
            landed = completed_view(s)
            p = 1.0
            for j in range(5):
                if intercept_status(s, j) in ('direct', 'approach'):
                    pj = float(p_success(ctx.off[s.pending[0]].r['pass'] - ctx.dfn[j].r['intercept'], ctx.rules))
                    p = min(p, pj)
            if s.inbound and s.pending[0] == ctx.inbounder:
                from .resolve import complete_pass
                landed = complete_pass(s)
            lost = s.with_(pending=None, approach=-1, tos=s.tos + 1)
            if p >= 1.0:
                return self(landed)
            if ctx.hold_for_one:
                lost_v = tuple(Val(0.0) for _ in self.targets)
            else:
                lost_v = self(lost)
            return mix([(p, self(landed)), (1 - p, lost_v)])
        budget_x = min(2, s.tok_x) if s.tok_x > 0 else 0
        # the shot already declared: the defence's reaction is all that is left
        if s.stage == 'reaction':
            maps = self.reach(s, budget_x)
            ev, look, p30, _ = self.defended(s, maps, s.ball, s.o[s.ball], s.shot, budget_x)
```

```
            return self._vals(s, base, ev, look, p30, s.ball, 0.0, 0.0)
        # clock about to run out: forced shot from where the carrier stands
        last_turn = s.inbound == 0 and s.clock <= RC.PHASE_SECONDS and (
            (s.side == 'X' and s.final) or (s.side == 'O' and s.tok_o == 0))
        if last_turn:
            if ctx.hold_for_one:
                return tuple(Val(0.0, 0.0, 0.0, 1.0, 0.0) for _ in self.targets)
            ev, look, p30, _, _ = self.carrier_look(s, s.ball, budget_x, False, forced=True)
            return self._vals(s, base + RC.TURNOVER_POINTS, ev, look, p30, s.ball, 1.0, ev)
        o_tokens = s.tok_o
        can_move = o_tokens >= 2 or s.side == 'X'
        if s.inbound and s.ball == ctx.inbounder:
            w, cands = self.cfg.inbound_kick_weight, [i for i in range(5) if i != s.ball and B.on_board(s.o[i])]
            # the ball still has to come in: weigh each receiver's look by the pass's interception risk
            best = (RC.TURNOVER_POINTS * 1.0, 0.0, 0.0, -1)
            can_pass = s.count('O', 'pass') < RC.LIMITS['O']['pass'] and s.tok_o > 0
            for r in cands if can_pass else []:
                ev, look, p30, _, _ = self.carrier_look(s, r, budget_x, True)
                pend = s.with_(pending=(s.ball, r))
                p = 1.0
                for j in range(5):
                    if intercept_status(pend, j) in ('direct', 'approach'):
                        p = min(p, float(p_success(ctx.off[s.ball].r['pass'] - ctx.dfn[j].r['intercept'], ctx.rules)))
                v = (p * ev + (1 - p) * RC.TURNOVER_POINTS) * w
                if v > best[0]:
                    best = (v, p * look * w, p * p30 * w, r)
            return self._vals(s, base, best[0], best[1], best[2], best[3], 0.0, 0.0)
        ev, look, p30, _, _ = self.carrier_look(s, s.ball, budget_x, can_move)
        shooter = s.ball
        if self.cfg.kick_weight > 0:
            for r in range(5):
                if r != s.ball and B.on_board(s.o[r]):
                    e2, l2, p2, _, _ = self.carrier_look(s, r, budget_x, True)
                    if e2 * self.cfg.kick_weight > ev:
                        ev, look, p30, shooter = e2 * self.cfg.kick_weight, l2 * self.cfg.kick_weight, p2 * self.cfg.kick_weight, r
        return self._vals(s, base, ev, look, p30, shooter, 0.0, 0.0)

    def _one(self, s):
        """One-turn evaluation (no recursion), cached."""
        k = ('one', s)
        v = self.cache.get(k)
        if v is None:
            v = self._eval(s)
            self.cache[k] = v
        return v

    def _two_turns(self, s, v1):
        """Best look over the offence's next two turns: a setup play now (no shot), the defence's
        best reply among its ball-side plays, then the one-turn evaluation."""
        gen = getattr(self, '_gen', None)
        if gen is None:
            gen = self._gen = Generator(self.cfg, self)
        pri = {'kick': 0, 'relocate+kick': 1, 'drive': 2, 'seal': 3, 'relocate': 4, 'screen': 5, 'swing': 6,
               'dribble': 7, 'hesitate': 8}
        setups = [p for p in gen.offence_plays(s) if not any(a.kind == 'shot' for a in p.actions)]
        setups.sort(key=lambda p: pri.get(p.concept, 9))
        best = None
        tried = 0
        for p in setups:
            if tried >= self.cfg.eval_setup_k:
                break
            node = apply_play(s, p)
            if node is None:
                continue
            tried += 1
            val = expected(node, self._reply)
            if best is None or val[0].ev > best[0].ev:
                best = val
        return best

    def _reply(self, s):
        """Defence's best reply (ball-side plays), then the one-turn evaluation."""
        if s.stage != 'play' or s.side != 'X':
            return self._one(s)
        gen = self._gen
        res = []
        for p in gen.plays(s, False, light=True)[:self.cfg.eval_reply_k + 2]:
            node = apply_play(s, p)
            if node is not None:
                res.append(expected(node, self._one))
        if not res:
            return self._one(s)
        res.sort(key=lambda vs: vs[0].ev)
        return res[0]

    def _terms(self, s, v):
        c = self.cfg
        if not (c.term_open_lane or c.term_zone) or s.stage != 'play' or s.side != 'O':
            return v
        extra = 0.0
        if c.term_zone and B.on_board(s.o[s.ball]) and B.zone(s.o[s.ball]) != '.':
            extra += c.term_zone
        if c.term_open_lane:
            mine = v[0].look
            best = mine
            for r in range(5):
                if r == s.ball or not B.on_board(s.o[r]):
```

```
                    continue
                pend = s.with_(pending=(s.ball, r))
                if any(intercept_status(pend, j) for j in range(5)):
                    continue
                best = max(best, self.carrier_look(s, r, min(2, s.tok_x), True)[0])
            extra += c.term_open_lane * max(0.0, best - mine)
        return tuple(Val(x.ev + extra, x.look, x.p30, x.pexp, x.fs) for x in v)

    def _vals(self, s, base, ev, look, p30, shooter, pexp, fs):
        out = []
        for t in self.targets:
            coach = 0.0
            tgt = s.target if t is None else t
            if s.ctx.coach_call and tgt == shooter:
                coach = RC.COACH_BONUS[band(int(round(look)))]
            total = 0.0 if (s.zeroed) else base + ev + coach
            out.append(Val(total, look, p30, pexp, fs))
        return tuple(out)


# ============================================================================
# plays (the candidate generator)
# ============================================================================

@dataclass(frozen=True)
class Play:
    concept: str
    actions: tuple

    def text(self):
        return ' then '.join(str(a) for a in self.actions if a.kind != 'end_turn') or str(self.actions[0])


def _contest_arrow(rules, q, sp):
    """The arrow a defender arriving on q with a Move sets to contest a shooter on sp: facing him, and
    under the step 3h geometries the wedge with the best contest category."""
    if rules.contest_geometry == 'current' and rules.arrow_mode == 'single':
        return B.dir_between(q, sp)
    bt = G.best_terms(rules, q, None, sp)
    return bt[4] if bt else B.dir_between(q, sp)


def _seq_actions(d, seq, arrow):
    """Turn a reach sequence into engine actions (the Move's arrow set to `arrow`)."""
    out = []
    for item in seq:
        kind, q = item[0], item[1]
        if kind == 'move':
            out.append(Action('move', 'X', d, q, arrow if arrow is not None else 0))
        elif len(item) > 2:                                  # a wedge Sprint's chosen other edge
            out.append(Action(kind, 'X', d, q, item[2]))
        else:
            out.append(Action(kind, 'X', d, q))
    return out


def quick_legal(s, a):
    """Cheap legality check for an action in state s."""
    if a.kind == 'end_turn':
        from .actions import inbound_held
        return s.turn_n >= 1 or s.stage == 'reaction' or (s.pending and s.side == 'X' and not inbound_held(s))
    if a.kind in MOVEMENT:
        if not can_move_kind(s, a.side, a.piece, a.kind):
            return False
        start = (s.o if a.side == 'O' else s.x)[a.piece]
        if a.kind == 'move' and a.dest == start:
            return a.side == 'X' and a.arrow != s.arrows[a.piece]
        return a.dest in movement_dests(s, a.side, a.piece, a.kind, s.ctx.rules)
    if a.kind == 'steal':
        cp = s.o[s.ball]
        return (s.tok_x > 0 and s.count('X', 'steal') < RC.LIMITS['X']['steal']
                and G.faces(s.ctx.rules, s.arrows[a.piece], s.x[a.piece], cp))
    return a in legal_actions(s)


def apply_play(s, play):
    """Apply a play's actions in order; returns a state or ChanceNode, or None if illegal."""
    node = s
    for i, a in enumerate(play.actions):
        if isinstance(node, ChanceNode) or node.stage == 'over':
            return None
        if i > 0 and (node.side != a.side or not quick_legal(node, a)):
            return None                    # the turn passed (e.g. its action limit), or the action is illegal
        node = apply(node, a)
    if not isinstance(node, ChanceNode) and node.stage in ('play', 'reaction') and node.side == s.side \
            and node.turn_n >= 1 and node.stage == s.stage:
        node = apply(node, Action('end_turn', s.side))
    return node


class Generator:
    def __init__(self, cfg, ev):
        self.cfg = cfg
        self.ev = ev

    def plays(self, s, shallow, rng=None, light=False):
        if s.stage == 'target':
```

```
            out = [Play('target', (a,)) for a in legal_actions(s)]
        elif s.stage in ('pick', 'contest', 'forced'):
            out = [Play(s.stage, (a,)) for a in legal_actions(s)]
        elif s.stage == 'reaction':
            out = self.reaction_plays(s)
        elif s.side == 'O':
            out = self.offence_plays(s)
        else:
            out = self.defence_plays(s, light)
        if shallow and self.cfg.top_k_raw > 0 and s.stage in ('play', 'reaction'):
            out += self.raw_plays(s, out)
        seen, uniq = set(), []
        for p in out:
            if p.actions not in seen:
                seen.add(p.actions)
                uniq.append(p)
        if rng is not None:
            rng.shuffle(uniq)
        return uniq

    def _value_after(self, s, play):
        node = apply_play(s, play)
        if node is None:
            return None
        return expected(node, self.ev)[0].ev

    def raw_plays(self, s, have):
        firsts = {p.actions[0] for p in have}
        scored = []
        maximise = s.side == 'O' and s.stage != 'reaction'
        for a in legal_actions(s):
            if a in firsts:
                continue
            p = Play('raw', (a,))
            v = self._value_after(s, p)
            if v is not None:
                scored.append((v if maximise else -v, p))
        scored.sort(key=lambda x: -x[0])
        return [p for _, p in scored[:self.cfg.top_k_raw]]

    # ---- offence ----
… lines 695–761: printed in the offence document …
    # ---- defence ----
… lines 764–922: printed in the defence document …

… lines 924–946: printed in the defence document …

… lines 948–975: printed in the defence document …
def expected(node, fn):
    """Expected Val tuple of a node under fn (state -> Val tuple)."""
    if isinstance(node, ChanceNode):
        return mix([(float(p), expected(sub, fn)) for p, _, sub in node.outcomes])
    return fn(node)


# ============================================================================
# exact solver
# ============================================================================
… lines 989–990: printed in the appendix …

… lines 993–1002: printed in the appendix …
REACH_BOUND = 1 + RC.MOVE_STRAIGHT + 2 + 2   # adjacent square + longest Sprint (3 + QS 2) + margin = 8 (>= 6 needed)
… lines 1008–1038: printed in the appendix …

… lines 1041–1083: printed in the appendix …
# ============================================================================
# guided search
# ============================================================================
… lines 1090–1094: printed in the appendix …

… lines 1097–1261: printed in the appendix …
```

playmaker_engine/solver_config.py

Shared: every width and weight the evaluation uses.

```
"""Every limit, width, depth, weight and verdict threshold the solver uses."""
from dataclasses import dataclass, replace


@dataclass(frozen=True)
class SolverConfig:
    # ---- exact endgame solver ----
    endgame_tokens: int = 2          # N: positions in the final phase with <= N tokens per side go exact
    exact_node_cap: int = 4000       # give up on an exact solve after this many applies (fall back to guided)
    # ---- candidate generator ----
    top_k_raw: int = 8               # K: best raw actions (by evaluation) added to the concept plays
    raw_depth: int = 1               # add raw actions only this many plies from the root
    drive_k: int = 3                 # drive / drive-and-shoot squares per node
    relocate_k: int = 2              # off-ball relocations per node
    closeout_k: int = 4              # closeouts per node (outside the reaction to a shot)
    combo_k: int = 3                 # defence two-action combos built from the top single plays
    contain_k: int = 2               # the carrier's best step-then-shoot squares the defence plays to contain
    # ---- guided search ----
```

```
    beam: int = 6                    # plays kept per node below the root (ordered by evaluation)
    root_beam: int = 40              # plays kept at the root
    max_depth: int = 12              # iterative deepening limit, in turns
    # ---- evaluation ----
    eval_turns: int = 2              # the evaluation looks over the offence's next N turns (1 or 2)
    eval_setup_k: int = 4            # offence setup plays (pass / relocate / drive...) tried in turn 1
    eval_reply_k: int = 4            # defence replies tried to each setup
    # small positional terms (kept only if they improve agreement with the exact solver; see report)
    term_open_lane: float = 0.0      # per point of the best open-lane receiver's look above the carrier's
    term_zone: float = 0.0           # bonus if the carrier stands on a shooting zone (not plain wood)
    quiescence: bool = True          # a leaf where the defence is to move gets one more defence turn
    quiescence_beam: int = 12        # defence plays kept in that extra turn (ball-side plays only, no combos)
    kick_weight: float = 0.0         # value of the best teammate look (0 = carrier only); tested in the report
    inbound_kick_weight: float = 1.0 # during an inbound the ball must go in: use the best receiver's look
    # ---- budgets and confidence ----
    budgets_min: tuple = (2.0, 10.0) # time budgets per card, minutes
    seeds: int = 5
    unstable_spread: float = 2.0     # value spread (points) above which a card is unstable
    # ---- verdicts ----
    gift_p30: float = 0.75
    wall_p30: float = 0.15
    wall_look: float = 20.0
    plan_share: float = 0.90         # a plan counts if it reaches this share of the best value
    goal: int = 30


DEFAULT = SolverConfig()


def with_config(**kw):
    return replace(DEFAULT, **kw)
```

computer_levels.json (shared keys)

Thinking time, level names.

```
{
  "_about": "How the computer chooses among legal actions (step 3e). Levels are presets of the levers below. No level gets hidden information, extra tokens or the Coach's Call target. Edit freely: computer.py reads everything from here.",
  "budget_s": {
    "server": 2.0,
    "browser": 2.0
  },
  "_budget_about": "Thinking time per decision (raised in step 3f: the designer prefers a slower, better choice). The browser build (Python compiled to the web) runs several times slower; the best choice found so far is used when time is up.",
  "names": {
    "rookie": [
      "Rookie",
      "makes beginner mistakes you can learn to spot"
    ],
    "starter": [
      "Starter",
      "solid, with one habit to find"
    ],
    "allstar": [
      "All-Star",
      "plays the full read"
    ]
  },
  "habit_words": {
    "always_force_middle": "always pointed the on-ball arrow at the baseline, shading drives to the middle",
    "nearest_closes_out": "always closed out with the nearest defender",
    "never_leaves_shooter": "never left a ▲ three-point shooter, even when the look was poor",
    "ball_watcher": "turned every help defender's arrow to the ball",
    "jumps_first_pass": "tried to intercept every pass it could",
    "settles_early": "lowered its shot bar a phase early, taking looks it could have waited on",
    "settles_for_first_shot": "took the first look above a low bar",
    "never_screens": "never set a screen",
    "ignores_mismatch": "passed by position order, not by matchup",
    "star_only": "always worked toward its highest-rated scorer",
    "forces_passes": "passed to its primary target whenever it could, however well the lane was guarded"
  },
  "concept_words": {
    "reaction": "didn't count the reaction to a declared shot",
    "reserve": "didn't keep a reaction in hand",
    "lookahead": "looked only at the shot on the board now",
    "help": "didn't send help off the ball",
    "walls": "didn't wall drives with the on-ball arrow",
    "lanes": "didn't point arrows across passing lanes",
    "plan": "didn't build toward a target",
    "patience": "didn't weigh waiting for a better shot",
    "drain": "didn't work to drain the defence's reactions",
    "risk": "ignored turnover risk"
  },
  "_tuned": "Head-to-head calibration (step 3e revised): see README 'How the computer plays' for every change and why."
}
```

Part 2. The defence

playmaker_engine/computer.py

The defence: choosing a defence play, the comfort line, the concepts and habits, contests and reactions, the threat of a dribble, the setup.

```
    def _threat(self, s):
        """The defence's view of the offence's next turn: its best action (a shot, a step and a shot, a pass
```

```
        and a shot), or a dribble at the dice, with the defence's reply after it (a won dribble can open a
        step-out shot a turn later)."""
        from .solver import expected
        v = self.ev(s)[0].ev
        if s.stage != 'play' or s.side != 'O' or s.tok_o < 1:
            return v
        for a in legal_actions(s):
            if a.kind == 'dribble':
                v = max(v, expected(apply(s, a), self.ev._reply)[0].ev)
        return v
```

… lines 521–564: printed in the shared core (Part 1) …

```
    def _defend(self, view, acts):
        from .solver import Play
        h = self.habits
        if 'jumps_first_pass' in h:
            icpt = [a for a in acts if a.kind == 'intercept']
            if icpt:
                self._fire('jumps_first_pass')
                o = view.ctx.off[view.pending[0]]
                return Play('habit', (min(icpt, key=lambda a: o.r['pass'] - view.ctx.dfn[a.piece].r['intercept']),))
        if view.stage == 'contest':
            return self._contest(view, acts)
        if view.stage == 'reaction' and 'nearest_closes_out' in h:
            p = self._nearest_reaction(view, acts)
            if p:
                self._fire('nearest_closes_out')
                return p
        if self.cfg['lookahead'] == 'solver' and view.stage == 'play':
            p = self._search(view)
            if p is not None:
                return self._habit_rewrite(view, p)
        plays = self._with_end(view, acts, self.gen.plays(view, shallow=False))
        if view.stage in ('reaction', 'contest') or view.pending:
            have = {p.actions for p in plays}
            plays += [Play('single', (a,)) for a in acts if (a,) not in have]
        plays = self._concept_filter(view, plays)
        fn = self._value_fn()
        if self._late():
            fn = self._immediate
        scored = self._rank(view, plays, fn, False, self._stop())
        if not scored:
            return None
        comfort = self._comfort_key(view)
        if comfort is not None:
            cost = lambda p: sum(a.kind != 'end_turn' for a in p.actions) * 1e-6
            with_line = min(scored, key=comfort)[1]
            without = min(scored, key=lambda vp: vp[0] + cost(vp[1]))[1]
            if with_line.actions != without.actions:
                self.stats['comfort_decided'] += 1          # the comfort line changed this choice
        best = self._pick(scored, False, key=comfort)
        return self._habit_rewrite(view, best)
```

… lines 605–605: printed in the shared core (Part 1) …

```
    def _comfort_key(self, view):
        if not self.cfg['concepts'].get('help', True) or self.comfort is None:
            return None
        line = self.comfort
        shooters = [i for i, c in enumerate(view.ctx.off) if c.bands['shot_three'] == 'H'] \
            if 'never_leaves_shooter' in self.habits else []
        cost = lambda p: sum(a.kind != 'end_turn' for a in p.actions) * 1e-6

        def key(vp):
            v, p = vp
            k = max(v, line)                    # looks below the comfort line are conceded
            if shooters and v < line:
                k += 1e-3 * v                   # ...except a ▲ shooter's (the habit): still guarded
            return k + cost(p)
        return key
```

… lines 621–621: printed in the shared core (Part 1) …

```
    def _concept_filter(self, view, plays):
        """Concepts off: no help (off-ball defenders stay by their man), no walls (the on-ball arrow stays
        on the ball), no lanes (off-ball arrows face their man)."""
        c = self.cfg['concepts']
        if c.get('help', True) and c.get('walls', True) and c.get('lanes', True):
            return plays
        from .solver import Play
        cp = view.o[view.ball]
        out, seen = [], set()
        for p in plays:
            acts, ok = [], True
            for a in p.actions:
                if a.side == 'X' and a.kind == 'move' and a.dest is not None:
                    on_ball = B.adjacent(a.dest, cp)
                    man = min((o for o in view.o), key=lambda o: B.cheb(o, a.dest))
                    if on_ball and not c.get('walls', True):
                        a = a._replace(arrow=B.nearest_dir(a.dest, cp))
                    elif not on_ball:
                        if not c.get('help', True) and B.cheb(a.dest, man) > 1:
                            ok = False
                        if not c.get('lanes', True):
                            a = a._replace(arrow=B.nearest_dir(a.dest, man))
                acts.append(a)
            if ok and tuple(acts) not in seen:
                seen.add(tuple(acts))
                out.append(Play(p.concept, tuple(acts)))
        return out or plays
```

… lines 649–649: printed in the shared core (Part 1) …

```
    def _habit_rewrite(self, view, play):
        """ball_watcher: help defenders' arrows to the ball; always_force_middle: the on-ball arrow points
        to the baseline."""
        h = self.habits
        if not ('ball_watcher' in h or 'always_force_middle' in h):
            return play
        from .solver import Play
        cp = view.o[view.ball]
        acts = []
        for a in play.actions:
            if a.side == 'X' and a.kind == 'move' and a.dest is not None and B.on_board(cp):
                if B.adjacent(a.dest, cp) and 'always_force_middle' in h:
                    if a.arrow != 4:
                        self._fire('always_force_middle')
                    a = a._replace(arrow=4)                  # S: toward the baseline
                elif not B.adjacent(a.dest, cp) and 'ball_watcher' in h:
                    d = B.nearest_dir(a.dest, cp)
                    if a.arrow != d:
                        self._fire('ball_watcher')
                    a = a._replace(arrow=d)
            acts.append(a)
        return Play(play.concept, tuple(acts))
```

… lines 672–672: printed in the shared core (Part 1) …

```
    def _contest(self, view, acts):
        from .solver import Play
        from .scoring import contest_options
        if 'nearest_closes_out' in self.habits:
            sp = view.o[view.ball]
            real = [a for a in acts if a.piece >= 0]
            if real:
                self._fire('nearest_closes_out')
                return Play('habit', (min(real, key=lambda a: (B.cheb(view.x[a.piece], sp), a.piece)),))
        scored = [(self._node_value(apply(view, a), lambda s: self.ev(s)[0].ev), Play('contest', (a,))) for a in acts]
        return self._pick(scored, False)
```

… lines 684–684: printed in the shared core (Part 1) …

```
    def _nearest_reaction(self, view, acts):
        """nearest_closes_out in the reaction: the nearest defender turns or steps to face the shooter."""
        from .solver import Play
        sp = view.o[view.ball]
        near = min(range(5), key=lambda j: (B.cheb(view.x[j], sp), j))
        for a in acts:
            if a.kind == 'move' and a.piece == near and B.adjacent(a.dest, sp) and a.arrow == B.dir_between(a.dest, sp):
                return Play('habit', (a,))
        return None
```

… lines 694–1342: printed in the shared core (Part 1) and the offence document …

```
def _next_to(o_sq, taken):
    """The empty square next to an attacker that is closest to the rim (ties: compass order)."""
    cands = [B.add(o_sq, d) for d in B.DIRS]
    cands = [q for q in cands if B.on_board(q) and q not in taken]
    return min(cands, key=lambda q: (B.dist_rim(q), cands.index(q))) if cands else None
```

… lines 1348–1349: printed in the shared core (Part 1) …

```
def place_defence(comp, o_squares, ball, build, budget_s=None):
    """The computer's defensive setup.
    1. Match up by number, unless a switch puts the best on-ball defender (Agility) on the most explosive
       creator (Quick Step, then Dribble), or the best three-point contester on the ▲ shooter, and the
       evaluation says the switch is worth it.
    2. Each defender starts next to their man; the on-ball defender's arrow is on the carrier unless
       walling the carrier's best drive is worth more.
    3. Everyone else: square (next to their man, or sagging up to two squares off) and arrow (their man,
       the ball, or the rim) chosen by the same evaluation as in play: help gain against leave cost,
       with the level's concepts and comfort line.
    build(x_squares, arrows) -> the starting GameState. Returns (x_squares, arrows, matchup)."""
    stop = time.time() + (budget_s if budget_s is not None else max(1.0, 2 * (comp.budget_s or 1.0)))
    o_squares = [tuple(q) for q in o_squares]
    probe = build(*_assign(o_squares, list(range(5))))
    comp._context(probe)
    ctx = comp.seen
    c = comp.cfg['concepts']

    def value(xs, arrows):
        s = build(xs, arrows)
        comp._context(s)
        v = comp._immediate(s) if comp.cfg['lookahead'] == 0 else comp.ev(s.with_(ctx=comp.seen))[0].ev
        return max(v, comp.comfort) + 1e-3 * v if (comp.comfort is not None and c.get('help', True)) else v

    perm = list(range(5))
    creator = max(range(5), key=lambda i: (ctx.off[i].qs, ctx.off[i].r['dribble'], -i))
    stopper = max(range(5), key=lambda j: (ctx.dfn[j].r['agility'], -j))
    shooters = [i for i in range(5) if ctx.off[i].bands['shot_three'] == 'H']
    closer = max(range(5), key=lambda j: (ctx.dfn[j].r['con_three'], -j))
    best_perm, best_v = perm, value(*_assign(o_squares, perm))
    for man, d in [(creator, stopper)] + [(i, closer) for i in shooters[:1]]:
        if perm[d] == man:
            continue
        trial = list(best_perm)
        k = trial.index(man)
        trial[d], trial[k] = trial[k], trial[d]
        v = value(*_assign(o_squares, trial))
        if v < best_v - 0.25:
            best_perm, best_v = trial, v
    xs, arrows = _assign(o_squares, best_perm)
    cp = o_squares[ball]
    # the on-ball defender: on the carrier, or a wall
    jb = best_perm.index(ball)
    if c.get('walls', True):
        for d in range(8):
```

```
            if time.time() > stop:
                break
            a2 = arrows[:jb] + [d] + arrows[jb + 1:]
            v = value(xs, a2)
            if v < best_v - 1e-9:
                best_v, arrows = v, a2
    # everyone else
    for j in range(5):
        if j == jb or time.time() > stop:
            continue
        man = o_squares[best_perm[j]]
        squares = [xs[j]]
        if c.get('help', True):
            taken = set(o_squares) | {q for k, q in enumerate(xs) if k != j}
            for dx in range(-2, 3):
                for dy in range(-2, 3):
                    q = (man[0] + dx, man[1] + dy)
                    if B.on_board(q) and q not in taken and q not in squares and B.dist_rim(q) <= B.dist_rim(man) + 1:
                        squares.append(q)
        for q in squares:
            if time.time() > stop:
                break
            faces = {B.nearest_dir(q, man)}
            if c.get('lanes', True):
                faces |= {B.nearest_dir(q, cp), B.nearest_dir(q, RC.RIM)}
            if 'ball_watcher' in comp.habits:
                faces = {B.nearest_dir(q, cp)}
            for d in faces:
                xs2 = xs[:j] + [q] + xs[j + 1:]
                a2 = arrows[:j] + [d] + arrows[j + 1:]
                try:
                    v = value(xs2, a2)
                except Exception:
                    continue
                if v < best_v - 1e-9:
                    best_v, xs, arrows = v, xs2, a2
    return xs, arrows, best_perm
```

… lines 1432–1433: printed in the shared core (Part 1) …

```
def _assign(o_squares, perm):
    """Defender j next to attacker perm[j] (closest to the rim), arrow on that attacker."""
    taken = set(o_squares)
    xs, arrows = [None] * 5, [None] * 5
    for j in sorted(range(5), key=lambda j: perm[j]):
        o = o_squares[perm[j]]
        q = _next_to(o, taken)
        taken.add(q)
        xs[j] = q
        arrows[j] = B.dir_between(q, o)
    return xs, arrows
```

playmaker_engine/defence.py

Arrow jobs (on the ball, wall, lane, help) and the board-reading defence from step 3e (kept for the arrow-job labels and its tests).

```
"""The computer defence that reads the board (step 3e).

The greedy baseline scores a defensive action by the carrier's shot *right now*, judged from the current
arrow facings. That makes "arrow on the shooter" win almost every time, although any defender with their
Move and a token left can turn (or step in) to face the shooter as a reaction once the shot is declared.

This defence scores each candidate turn with the solver's evaluation instead, which already counts
reaction-reachable contests (a defender whose Move is unused, who is not locked, with a token in the
pool, can still get adjacent and facing as a reaction) and looks one offence turn ahead (shoot now,
drive, or pass and then shoot). Walls and lanes start to score, and holding a reaction back is valued.

It plays only from the legal actions, reads only the board and the cards (what a human sees), and never
looks at the offence's secret Coach's Call target.
"""
from . import board as B
from . import geometry as G
from .actions import legal_actions
from .policies import Policy, GreedyPolicy, expected
from .resolve import apply, ChanceNode


def arrow_job(state, j):
    """What defender j's arrow is doing, in the rules' four jobs (for logs, tests and the review):
    'on_ball' (on the ball carrier: Steal and a clean contest), 'wall' (adjacent to the carrier, line
    across the carrier's way to the rim), 'lane' (the line crosses a pass from the carrier to a teammate),
    'help' (the line crosses the carrier's drive path to the rim), 'on_man' (on the nearest other
    attacker) or 'other'."""
    xp, a = state.x[j], state.arrows[j]
    cp = state.o[state.ball]          # off the court during an inbound: only the lanes matter then
    inside = B.on_board(cp)
    R = state.ctx.rules
    if G.faces(R, a, xp, cp):
        return 'on_ball'
    rim = B.RIM
    if inside and B.adjacent(xp, cp) and G.crosses(R, xp, a, cp, rim):
        return 'wall'
    for i, op in enumerate(state.o):
        if i != state.ball and B.on_board(op) and G.crosses(R, xp, a, cp, op):
            return 'lane'
    if inside and G.crosses(R, xp, a, cp, rim):
        return 'help'
    others = [op for i, op in enumerate(state.o) if i != state.ball and B.on_board(op)]
    if others:
```

```
        near = min(others, key=lambda op: B.cheb(op, xp))
        if B.adjacent(xp, near) and a == B.dir_between(xp, near):
            return 'on_man'
    return 'other'


DEFAULT_BUDGET_S = 2.5          # thinking time per decision (the in-browser build sets a lower one)


class ReadingDefence(Policy):
    """Defence: the candidate turn (the solver's defence plays, plus every single legal action for the
    reaction and the contest choice) with the lowest value under the solver's two-turn evaluation.
    Offence: the greedy baseline (unchanged)."""
    name = 'reader'

    def __init__(self, eval_turns=1, budget_s='default'):
        budget_s = DEFAULT_BUDGET_S if budget_s == 'default' else budget_s
        self.budget_s = budget_s      # thinking time per decision; the best play so far is used after that
        from .solver import Evaluator, Generator
        from .solver_config import with_config
        self.cfg = with_config(eval_turns=eval_turns)
        self._Evaluator, self._Generator = Evaluator, Generator
        self._ctx = None
        self._fresh()
        self.offence = GreedyPolicy()

    def _fresh(self):
        # the solver's caches key positions without their context (teams, rules): one set per context
        self.ev = self._Evaluator(self.cfg)
        self.gen = self._Generator(self.cfg, self.ev)
        self.plan = None

    def _context(self, state):
        if state.ctx is not self._ctx:
            self._ctx = state.ctx
            self._fresh()

    def reset(self, seed):
        self.offence.reset(seed)
        self.plan = None

    def choose(self, state):
        acts = legal_actions(state)
        if len(acts) == 1:
            return acts[0]
        defending = state.stage in ('contest', 'reaction') or (state.stage == 'play' and state.side == 'X')
        if not defending:
            return self.offence.choose(state)
        self._context(state)
        if self.plan and self.plan[0] == state and self.plan[1]:
            a = self.plan[1][0]
            if a in acts:
                self._advance(state, a, self.plan[1][1:])
                return a
        self.plan = None
        best = self.best_play(state, acts)
        if best is None:
            return GreedyPolicy().choose(state)
        a = best.actions[0]
        self._advance(state, a, best.actions[1:])
        return a

    def _advance(self, state, a, rest):
        nxt = apply(state, a)
        self.plan = (nxt, rest) if rest and not isinstance(nxt, ChanceNode) else None

    def value(self, node):
        return expected(node, lambda s: self.ev(s)[0].ev)

    def best_play(self, state, acts=None):
        from .solver import Play, apply_play
        self._context(state)
        if state.ctx.coach_call and state.stage != 'over':
            state = state.with_(target=-1)        # the defence never sees the offence's secret target
        acts = acts if acts is not None else legal_actions(state)
        plays = self.gen.plays(state, shallow=False)
        if state.stage in ('reaction', 'contest') or state.pending:
            have = {p.actions for p in plays}
            plays += [Play('single', (a,)) for a in acts if (a,) not in have]
        import time
        stop = time.time() + self.budget_s if self.budget_s else None
        best, best_v = None, None
        for p in plays:
            if stop is not None and best is not None and time.time() > stop:
                break
            node = apply_play(state, p)
            if node is None:
                continue
            v = self.value(node)
            # spend a token only for a real gain: ties go to the play with fewer actions
            v += sum(a.kind != 'end_turn' for a in p.actions) * 1e-6
            if best_v is None or v < best_v:
                best, best_v = p, v
        return best


def choose_arrows(state, passes=2):
```

```
    """Setup arrows for a placed defence: start from each defender facing its man, then (coordinate
    descent) turn one defender at a time to the facing that leaves the offence the lowest value under the
    solver's evaluation. Squares are not changed."""
    from .solver import Evaluator
    from .solver_config import with_config
    ev = Evaluator(with_config(eval_turns=1))
    arrows = list(state.arrows)
    best = ev(state)[0].ev
    for _ in range(passes):
        changed = False
        for j in range(5):
            for d in range(8):
                if d == arrows[j]:
                    continue
                trial = arrows[:j] + [d] + arrows[j + 1:]
                v = ev(state.with_(arrows=tuple(trial)))[0].ev
                if v < best - 1e-9:
                    best, arrows, changed = v, trial, True
        if not changed:
            break
    return arrows
```

playmaker_engine/solver.py

The defence and reaction play generators: closeouts, help, deny, contain, recover / switch, combos; the reaction to a declared shot.

```
    def defence_plays(self, s, light=False):
        out = []
        rules = s.ctx.rules
        view = s
        if s.pending:
            from .actions import completed_view, is_approach
            acts = legal_actions(s)
            for a in acts:
                if a.kind == 'intercept':
                    out.append(Play('intercept', (a,)))
                elif a.kind in MOVEMENT and is_approach(s, a):
                    out.append(Play('approach+intercept', (a, Action('intercept', 'X', a.piece))))
            from .actions import inbound_held
            if s.inbound and s.pending[0] == s.ctx.inbounder:
                if not inbound_held(s):
                    out.append(Play('allow', (Action('end_turn', 'X'),)))
                    return out
                # OQ-35: the defence plays a normal turn while the inbound is in the air
                for a in acts:
                    if a.kind in MOVEMENT and is_approach(s, a):
                        for b in (Action('intercept', 'X', a.piece),):
                            out.append(Play('approach+intercept', (a, b)))
                view = s
            else:
                view = completed_view(s)
        acts = legal_actions(view)
        c = view.ball
        cp = view.o[c]
        budget = min(2, view.tok_x)
        maps = self.ev.reach(view, budget)
        singles = []
        for a in acts:
            if a.kind == 'steal':
                out.append(Play('steal', (a,)))
            elif a.kind == 'challenge':
                out.append(Play('challenge', (a,)))
            elif a.kind == 'pause':
                out.append(Play('pause', (a,)))
        # closeouts on the carrier (Move facing, then Quick Step in), strongest contest first
        cl = []
        for d in range(5):
            free, fixed = maps[d]
            for dr in B.DIRS:
                q = (cp[0] + dr[0], cp[1] + dr[1])
                seq = free.get(q)
                if seq is None or not B.on_board(q):
                    continue
                on = _contest_arrow(s.ctx.rules, q, cp)
                acts_seq = _seq_actions(d, seq, on)
                score_c = max((s.ctx.dfn[d].con(t) for t in RC.ZONE_SHOTS.get(B.zone(cp), ()) ), default=0)
                cl.append((score_c, -len(seq), d, acts_seq))
        cl.sort(key=lambda x: (-x[0], -x[1]))
        for _, _, d, seq in cl[:self.cfg.closeout_k]:
            out.append(Play('closeout', tuple(seq)))
            if len(seq) == 1:
                singles.append(('closeout', seq[0]))
        # help: step into the drive lane between the carrier and the rim
        for d in range(5):
            free, _ = maps[d]
            best = None
            for q, seq in free.items():
                if len(seq) != 1 or q == view.x[d]:
                    continue
                if 1 <= B.cheb(q, cp) <= 2 and B.dist_rim(q) < B.dist_rim(cp):
                    k = (B.dist_rim(q), B.cheb(q, cp))
                    if best is None or k < best[0]:
                        best = (k, q)
            if best:
                a = Action('move', 'X', d, best[1], B.nearest_dir(best[1], cp))
                out.append(Play('help', (a,)))
                singles.append(('help', a))
```

```
        # deny: put an arrow across a passing lane (turn in place, or step onto the lane)
        for r in range(5):
            if r == c or not B.on_board(view.o[r]):
                continue
            a_, b_ = cp, view.o[r]
            found = None
            for d in range(5):
                if not can_move_kind(view, 'X', d, 'move'):
                    continue
                xp = view.x[d]
                for arr in range(8):
                    if arr != view.arrows[d] and G.crosses(rules, xp, arr, a_, b_):
                        found = Action('move', 'X', d, xp, arr)
                        break
                if found:
                    break
            if found:
                out.append(Play('deny', (found,)))
                singles.append(('deny', found))
        # contain: the carrier's best looks after a step (Move, Quick Step or Sprint, then shoot) are one
        # offence turn away; stand next to those squares, facing them, before the turn passes
        if s.side == 'X' and B.on_board(cp) and not view.pending:
            for q in self.step_shots(view, c, budget, self.cfg.contain_k):
                cl = []
                for d in range(5):
                    free, fixed = maps[d]
                    for dr in B.DIRS:
                        r = (q[0] + dr[0], q[1] + dr[1])
                        if not B.on_board(r) or r in view.o:
                            continue
                        on = _contest_arrow(s.ctx.rules, r, q)
                        seq, clean = free.get(r), True
                        if seq is None and r in fixed:            # a step that keeps the arrow: facing q, or a foul roll
                            arrows = fixed[r]
                            fa = [k for k in arrows if G.faces(s.ctx.rules, k, r, q)]
                            if fa:
                                a = min(fa, key=lambda k: len(arrows[k]))
                            elif s.ctx.rules.contest_geometry != 'current':
                                continue                          # step 3h: no contest with the lines off him
                            else:
                                a = min(arrows, key=lambda k: len(arrows[k]))
                            seq, clean = arrows[a], a in fa
                        if not seq or len(seq) > view.tok_x:     # none, or already there
                            continue
                        con = max((s.ctx.dfn[d].con(t) for t in RC.ZONE_SHOTS.get(B.zone(q), ())), default=0)
                        cl.append((not clean, -con, len(seq), B.dist_rim(r), d, tuple(_seq_actions(d, seq, on))))
                cl.sort(key=lambda x: x[:5])
                used = set()
                for _, _, _, _, d, seq in cl:
                    if d in used:
                        continue
                    used.add(d)
                    out.append(Play('contain', seq))
                    if len(seq) == 1:
                        singles.append(('contain', seq[0]))
                    if len(used) >= 2:
                        break
        if light:                          # quiescence: plays around the ball only
            return out
        # recover / switch: back to a man, rim side, arrow on him
        for d in range(5):
            free, _ = maps[d]
            for kind, man in (('recover', d), ('switch', None)):
                if man is None:
                    others = [o for o in range(5) if o != d and B.on_board(view.o[o])]
                    if not others:
                        continue
                    man = min(others, key=lambda o: B.cheb(view.o[o], view.x[d]))
                    if man == d:
                        continue
                mp = view.o[man]
                if not B.on_board(mp) or B.adjacent(view.x[d], mp):
                    continue
                cands = [(B.dist_rim(q), q) for q, seq in free.items() if len(seq) == 1 and B.adjacent(q, mp)]
                if cands:
                    q = min(cands)[1]
                    a = Action('move', 'X', d, q, B.dir_between(q, mp))
                    out.append(Play(kind, (a,)))
                    singles.append((kind, a))
        # two-action combos from the best singles (different defenders)
        if view.tok_x >= 2:
            seen = 0
            for i, (k1, a1) in enumerate(singles):
                for k2, a2 in singles[i + 1:]:
                    if a1.piece != a2.piece and seen < self.cfg.combo_k * 3:
                        out.append(Play(f'{k1}+{k2}', (a1, a2)))
                        seen += 1
        return out
```

… lines 923–923: printed in the shared core (Part 1) …

```
    def step_shots(self, s, shooter, budget_x, k):
        """The k best squares (other than where the carrier stands) the carrier can step to with one
        movement action and shoot from, by the value of the shot against the defence as it stands."""
        if k <= 0:
            return []
        spos = s.o[shooter]
        maps = self.ev.reach(s, budget_x)
        squares = set()
```

```
        st = s.with_(side='O', ball=shooter)
        for kind in ('move', 'qs', 'sprint'):
            if can_move_kind(st, 'O', shooter, kind):
                squares |= set(movement_dests(st, 'O', shooter, kind, s.ctx.rules))
        scored = []
        for q in squares:
            if q == spos or not B.on_board(q):
                continue
            sh = s.with_(o=s.o[:shooter] + (q,) + s.o[shooter + 1:])
            best = max((self.ev.defended(sh, maps, shooter, q, t, budget_x)[0] for t in RC.ZONE_SHOTS[B.zone(q)]),
                       default=None)
            if best is not None:
                scored.append((best, q))
        scored.sort(key=lambda x: (-x[0], x[1]))
        return [q for _, q in scored[:k]]
```

… lines 947–947: printed in the shared core (Part 1) …

```
    def reaction_plays(self, s):
        out = [Play('no reaction', (Action('end_turn', 'X'),))]
        sp = s.o[s.ball]
        budget = min(2, s.tok_x)
        maps = self.ev.reach(s, budget)
        for a in legal_actions(s):
            if a.kind == 'steal':
                out.append(Play('steal gamble', (a,)))
        for d in range(5):
            free, fixed = maps[d]
            for dr in B.DIRS:
                q = (sp[0] + dr[0], sp[1] + dr[1])
                if not B.on_board(q):
                    continue
                on = _contest_arrow(s.ctx.rules, q, sp)
                seq = free.get(q)
                if seq is not None and len(seq) <= budget:
                    acts_seq = _seq_actions(d, seq, on)
                    out.append(Play('closeout', tuple(acts_seq)))
                    if len(acts_seq) == 1 and budget >= 2:
                        out.append(Play('closeout+steal', (acts_seq[0], Action('steal', 'X', d))))
                for arr, fseq in fixed.get(q, {}).items():
                    if fseq and len(fseq) <= budget:
                        facing = G.faces(s.ctx.rules, arr, q, sp)
                        if not facing and s.ctx.rules.contest_geometry != 'current':
                            continue                    # step 3h: lines off the shooter can't contest
                        out.append(Play('closeout (side-on)' if not facing else 'closeout', tuple(_seq_actions(d, fseq, None))))
        return out
```

computer_levels.json (defence)

The defence levels, habits and levers.

```
{
  "comfort_line": 15,
  "_comfort_about": "The defence concedes any look worth less than this (the Okay/Poor prestige boundary): it won't spend to push the of
fence below it. A level's comfort_offset is added.",
  "levels": {
    "rookie": {
      "concepts": {
        "reaction": true,
        "reserve": false,
        "lookahead": false,
        "help": false,
        "walls": false,
        "lanes": false
      },
      "habits": 2,
      "habits_not": [
        "jumps_first_pass"
      ],
      "tunnel": 2,
      "reading": "role",
      "comfort_offset": 6,
      "lookahead": 0,
      "randomness": 0.15
    },
    "starter": {
      "concepts": {
        "reaction": true,
        "reserve": true,
        "lookahead": true,
        "help": true,
        "walls": true,
        "lanes": true
      },
      "habits": 1,
      "tunnel": 4,
      "reading": "marks",
      "comfort_offset": 3,
      "lookahead": 1,
      "randomness": 0.05
    },
    "allstar": {
      "concepts": {
        "reaction": true,
        "reserve": true,
        "lookahead": true,
        "help": true,
        "walls": true,
        "lanes": true
```

```
      },
      "habits": 0,
      "tunnel": 5,
      "reading": "exact",
      "comfort_offset": 0,
      "lookahead": 1,
      "randomness": 0.0
    }
  },
  "habits": [
    "always_force_middle",
    "nearest_closes_out",
    "never_leaves_shooter",
    "ball_watcher",
    "jumps_first_pass"
  ],
  "_habits_not_about": "habits_not: habits a level never draws. Rookie: not jumps_first_pass, since at Pass 14 v Intercept 6 an intercept attempt succeeds 60% of the time, so jumping passes is often right, not a beginner's mistake.",
  "habit_words": {
    "always_force_middle": "always pointed the on-ball arrow at the baseline, shading drives to the middle",
    "nearest_closes_out": "always closed out with the nearest defender",
    "never_leaves_shooter": "never left a ▲ three-point shooter, even when the look was poor",
    "ball_watcher": "turned every help defender's arrow to the ball",
    "jumps_first_pass": "tried to intercept every pass it could"
  },
  "concept_words": {
    "reaction": "didn't count the reaction to a declared shot",
    "reserve": "didn't keep a reaction in hand",
    "lookahead": "looked only at the shot on the board now",
    "help": "didn't send help off the ball",
    "walls": "didn't wall drives with the on-ball arrow",
    "lanes": "didn't point arrows across passing lanes",
    "plan": "didn't build toward a target",
    "patience": "didn't weigh waiting for a better shot",
    "drain": "didn't work to drain the defence's reactions",
    "risk": "ignored turnover risk"
  }
}
```

Appendix

playmaker_engine/policies.py

The base player class, and the greedy baseline computer (the "baseline" option, not offered on the setup screen).

```python
"""Agents and the game loop. No rule logic lives here: policies only pick from legal_actions."""
import random
from fractions import Fraction

from .actions import legal_actions, MOVEMENT
from .resolve import apply, sample, ChanceNode
from .scoring import score, is_terminal, shot_ev


class Policy:
    """choose(state) -> one of legal_actions(state)."""
    name = 'policy'

    def reset(self, seed):
        pass

    def choose(self, state):
        raise NotImplementedError


class RandomPolicy(Policy):
    name = 'random'

    def __init__(self, seed=0):
        self.rng = random.Random(seed)

    def reset(self, seed):
        self.rng = random.Random(seed)

    def choose(self, state):
        return self.rng.choice(legal_actions(state))


def expected(node, fn):
    """Expected value of fn over a node's outcomes."""
    if isinstance(node, ChanceNode):
        return sum(Fraction(p) * expected(s, fn) for p, _, s in node.outcomes)
    return Fraction(fn(node))


def immediate_shot_value(state):
    """Best value the ball carrier could get by shooting right now (no defensive reaction)."""
    from . import board as B
    from . import rules_config as RC
    if state.stage == 'over':
        return score(state).total
    p = state.o[state.ball]
    if not B.on_board(p):
        return 0
    types = RC.ZONE_SHOTS[B.zone(p)]
    if not types:
```

```
        return 0
    return max(shot_ev(state, state.ball, t) for t in types)


class GreedyPolicy(Policy):
    """Baseline. Offence: take the best shot available this turn (shoot now, or one
    movement action then shoot); otherwise drift toward a better shot. Defence: answer
    a pass with the best intercept, then pick the action that leaves the carrier the
    worst immediate shot (and contest a declared shot as well as it can)."""
    name = 'greedy'

    def choose(self, state):
        acts = legal_actions(state)
        if len(acts) == 1:
            return acts[0]
        if state.stage == 'target':
            return max(acts, key=lambda a: max(state.ctx.off[a.target].r[k] for k in state.ctx.off[a.target].r if k.startswith('shot
_')))
        if state.stage == 'forced':
            return max(acts, key=lambda a: expected(apply(state, a), lambda n: min(
                (expected(apply(n, c), immediate_shot_value) for c in legal_actions(n)), default=Fraction(0))
                if n.stage == 'contest' else immediate_shot_value(n)))
        if state.side == 'O':
            return self._offence(state, acts)
        return self._defence(state, acts)

    def _offence(self, state, acts):
        shots = [a for a in acts if a.kind == 'shot']
        best_now = max(shots, key=lambda a: shot_ev(state, a.piece, a.shot)) if shots else None
        v_now = shot_ev(state, best_now.piece, best_now.shot) if best_now else None
        best_mv, v_mv = None, None
        for a in acts:
            if a.kind not in MOVEMENT or a.piece != state.ball:
                continue
            nxt = apply(state, a)
            if isinstance(nxt, ChanceNode) or nxt.side != 'O' or nxt.stage != 'play':
                continue
            v = immediate_shot_value(nxt)
            if v_mv is None or v > v_mv:
                best_mv, v_mv = a, v
        if best_now and (v_mv is None or v_now >= v_mv):
            return best_now
        if best_mv is not None:
            return best_mv
        if best_now:
            return best_now
        for kind in ('end_turn', 'hesitate', 'pass'):
            for a in acts:
                if a.kind == kind:
                    return a
        return acts[0]

    def _defence(self, state, acts):
        icpt = [a for a in acts if a.kind == 'intercept']
        if icpt:
            o = state.ctx.off[state.pending[0]]
            return min(icpt, key=lambda a: o.r['pass'] - state.ctx.dfn[a.piece].r['intercept'])
        best, best_v = None, None
        for a in acts:
            nxt = apply(state, a)
            v = expected(nxt, immediate_shot_value)
            if a.kind not in ('end_turn',):
                v += Fraction(1, 1000)        # spend tokens only for a real gain
            if best_v is None or v < best_v:
                best, best_v = a, v
        return best


POLICIES = {'random': RandomPolicy, 'greedy': GreedyPolicy}


def play(setup, offence_policy, defence_policy, seed=0, max_steps=5000, check=None):
    """Play one possession. setup: a GameState or a scenario number.
    Returns dict(log=[...], score=Score, final=GameState, states=n). Same seed, same game.
    check: optional callable(state) run after every apply (invariants)."""
    from .scenarios import scenario_state
    state = scenario_state(setup) if isinstance(setup, int) else setup
    rng = random.Random(seed)
    offence_policy.reset(seed * 2 + 1)
    defence_policy.reset(seed * 2 + 2)
    log = []
    steps = 0
    while not is_terminal(state):
        steps += 1
        if steps > max_steps:
            raise RuntimeError('game did not terminate')
        pol = offence_policy if state.side == 'O' or state.stage == 'target' else defence_policy
        a = pol.choose(state)
        before = state
        nxt = apply(state, a)
        label = ''
        if isinstance(nxt, ChanceNode):
            nxt, label = sample(nxt, rng)
        log.append((before.clock, before.phase, str(a), label))
        state = nxt
        if check:
            check(before, a, state)
```

```
    return dict(log=log, score=score(state), final=state, steps=steps)


def format_log(result):
    lines = []
    for clock, phase, a, label in result['log']:
        lines.append(f'[phase {phase}, clock {clock:>2}] {a}' + (f'  -> {label}' if label else ''))
    lines.append(f'Score: {result["score"]}')
    return '\n'.join(lines)
```

playmaker_engine/solver.py

The solver's search and the exact endgame solver. A computer level uses it only with lookahead "solver", which no level uses at present.

```
class NodeCap(Exception):
    pass
```

… lines 991–992: printed in the shared core (Part 1) …

```
def _terminal_turn(s):
    """Is this the defence's last turn before the possession resolves (contest or forced shot)?"""
    if not (s.side == 'X' and not s.pending and s.inbound == 0 and s.ctx.rules.contest_choice in ('min_value', 'player')):
        return False
    if s.stage == 'reaction':
        return True
    if s.stage != 'play' or s.clock > RC.PHASE_SECONDS:
        return False
    # the final turn, or the defence's last token (which ends the phase at once, then the clock runs out)
    return s.final or (s.tok_x == 1 and s.ctx.rules.defence_empty_ends_phase)
```

… lines 1003–1007: printed in the shared core (Part 1) …

```
def pruned_actions(s):
    """Legal actions, with provably irrelevant defence actions removed on its last turn.
    On that turn only the defenders next to the shooter, their arrows and Steals matter:
      * second actions that don't end a defender next to the shooter (or turn an adjacent
        defender's arrow, or Steal) can only remove contest options, never add one;
      * first actions by defenders that start and end far from the shooter can't affect it
        (they can't reach it, and can't unblock anyone who can); one is kept as a no-op."""
    acts = legal_actions(s)
    if not _terminal_turn(s):
        return acts
    sp = s.o[s.ball]
    keep, noop = [], None
    for a in acts:
        if a.kind in ('steal', 'end_turn', 'forfeit'):
            keep.append(a)
        elif a.kind in MOVEMENT:
            start = s.x[a.piece]
            if s.turn_n >= 1 or s.tok_x <= 1:          # this is the defence's last action
                if B.adjacent(a.dest, sp):
                    keep.append(a)
                elif noop is None and s.turn_n == 0 and not B.adjacent(start, sp):
                    noop = a                           # a true no-op: nobody near the shooter moves
            elif B.cheb(a.dest, sp) <= REACH_BOUND or B.cheb(start, sp) <= REACH_BOUND:
                keep.append(a)
            elif noop is None:
                noop = a
        elif s.turn_n == 0 and noop is None:
            noop = a
    if noop is not None and not any(a.kind == 'end_turn' for a in keep):
        keep.append(noop)
    return keep
```

… lines 1039–1040: printed in the shared core (Part 1) …

```
def exact_solve(s, cap=10 ** 6, prune=True, tt=None, stats=None):
    """Exact best play from s (target as set in s): expectimax over raw engine actions with a
    transposition table. Returns (Val, {action: Val}) for the root. Raises NodeCap past `cap` applies."""
    tt = {} if tt is None else tt
    count = [0]

    def node_value(node):
        if isinstance(node, ChanceNode):
            return mix([(float(p), (node_value(sub),)) for p, _, sub in node.outcomes])[0]
        return rec(node)

    def rec(st):
        if st.stage == 'over':
            return terminal_vals(st, (None,))[0]
        v = tt.get(st)
        if v is not None:
            return v
        acts = pruned_actions(st) if prune else legal_actions(st)
        vals = []
        for a in acts:
            count[0] += 1
            if count[0] > cap:
                raise NodeCap()
            vals.append(node_value(apply(st, a)))
        maximise = st.side == 'O' or st.stage in ('target', 'pick')
        v = max(vals, key=lambda x: x.ev) if maximise else min(vals, key=lambda x: x.ev)
        tt[st] = v
        return v

    if s.stage == 'over':
        return terminal_vals(s, (None,))[0], {}
    acts = pruned_actions(s) if prune else legal_actions(s)
    per = {}
    for a in acts:
        count[0] += 1
        if count[0] > cap:
```

```
            raise NodeCap()
        per[a] = node_value(apply(s, a))
    maximise = s.side == 'O' or s.stage in ('target', 'pick')
    v = max(per.values(), key=lambda x: x.ev) if maximise else min(per.values(), key=lambda x: x.ev)
    if stats is not None:
        stats['applies'] = count[0]
    return v, per
```

… lines 1084–1089: printed in the shared core (Part 1) …

```
@dataclass
class Entry:
    vals: tuple
    best: Play = None
    children: list = field(default_factory=list)     # [(Play, vals)]
```

… lines 1095–1096: printed in the shared core (Part 1) …

```
class Searcher:
    def __init__(self, cfg=DEFAULT, targets=(None,), seed=0, deadline=None):
        self.cfg = cfg
        self.targets = targets
        self.ev = Evaluator(cfg, targets)
        self.gen = Generator(cfg, self.ev)
        self.rng = random.Random(seed) if seed else None
        self.deadline = deadline
        self.tt = {}
        self.nodes = 0
        self.exact_hits = 0
        self.exact_tries = 0
        self.exact_tt = {}
        self.root_depth = 0
        self.concept_counts = {}

    def choose(self, children, maximise):
        """Offence: best per target (it knows its own target). Defence: the play that
        minimises the offence's best value across targets."""
        if maximise:
            if len(self.targets) == 1:
                p, v = max(children, key=lambda pv: pv[1][0].ev)
                return v, p
            out = []
            for t in range(len(self.targets)):
                out.append(max(children, key=lambda pv: pv[1][t].ev)[1][t])
            p = max(children, key=lambda pv: max(x.ev for x in pv[1]))[0]
            return tuple(out), p
        p, v = min(children, key=lambda pv: max(x.ev for x in pv[1]))
        return v, p

    def in_exact_range(self, s):
        n = self.cfg.endgame_tokens
        return (len(self.targets) == 1 and s.stage == 'play' and s.inbound == 0 and s.clock <= RC.PHASE_SECONDS
                and s.tok_o <= n and s.tok_x <= n and not s.ctx.coach_call)

    def value(self, s, depth):
        if s.stage == 'over':
            return terminal_vals(s, self.targets)
        if self.deadline and time.time() > self.deadline:
            raise Timeout()
        k = (s, depth)
        e = self.tt.get(k)
        if e is not None:
            return e.vals
        self.nodes += 1
        if self.in_exact_range(s):
            self.exact_tries += 1
            try:
                v, _ = exact_solve(s, cap=self.cfg.exact_node_cap, tt=self.exact_tt)
                self.exact_hits += 1
                vals = (v,)
                self.tt[k] = Entry(vals)
                return vals
            except NodeCap:
                pass
        if depth <= 0:
            if self.cfg.quiescence and s.side == 'X' and s.stage == 'play':
                return self.quiesce(s)
            return self.ev(s)
        shallow = (self.root_depth - depth) < self.cfg.raw_depth
        plays = self.gen.plays(s, shallow, self.rng)
        maximise = (s.side == 'O' and s.stage != 'reaction') or s.stage in ('target', 'pick', 'forced')
        children = []
        for p in plays:
            node = apply_play(s, p)
            if node is None:
                continue
            children.append((p, node))
        if not children:
            return self.ev(s)
        width = self.cfg.root_beam if depth == self.root_depth else self.cfg.beam
        if len(children) > width and depth >= 2:
            scored = [(expected(n, self.ev), p, n) for p, n in children]
            scored.sort(key=lambda x: (-x[0][0].ev if maximise else max(v.ev for v in x[0])))
            children = [(p, n) for _, p, n in scored[:width]]
        results = []
        for p, node in children:
            v = self.expect(node, depth - 1)
            results.append((p, v))
        vals, best = self.choose(results, maximise)
        self.tt[k] = Entry(vals, best, results)
        return vals
```

```python

    def quiesce(self, s):
        """Leaf with the defence to move: let it take its turn (best of its concept plays), then evaluate."""
        k = (s, -1)
        e = self.tt.get(k)
        if e is not None:
            return e.vals
        plays = self.gen.plays(s, False, light=True)
        results = []
        for p in plays:
            node = apply_play(s, p)
            if node is None:
                continue
            results.append((p, expected(node, self.ev)))
        if not results:
            vals = self.ev(s)
        else:
            results.sort(key=lambda pv: max(v.ev for v in pv[1]))
            vals, best = self.choose(results[:self.cfg.quiescence_beam], False)
        self.tt[k] = Entry(vals)
        return vals

    def expect(self, node, depth):
        if isinstance(node, ChanceNode):
            return mix([(float(p), self.expect(sub, depth)) for p, _, sub in node.outcomes])
        return self.value(node, depth)

    def search(self, s, budget_s=None, max_depth=None, min_depth=1):
        """Iterative deepening. Returns dict with vals, depth reached, best play, timings."""
        max_depth = max_depth or self.cfg.max_depth
        t0 = time.time()
        self.deadline = (t0 + budget_s) if budget_s else None
        done = None
        for d in range(1, max_depth + 1):
            self.root_depth = d
            try:
                vals = self.value(s, d)
            except Timeout:
                break
            done = (d, vals, self.tt.get((s, d)))
            if self.deadline is None and d >= max_depth:
                break
            if s.stage == 'over':
                break
        if done is None:                          # nothing finished: fall back to depth 1 without a deadline
            self.deadline = None
            self.root_depth = min_depth
            vals = self.value(s, min_depth)
            done = (min_depth, vals, self.tt.get((s, min_depth)))
        d, vals, entry = done
        return dict(depth=d, vals=vals, entry=entry, seconds=time.time() - t0, nodes=self.nodes,
                    evals=self.ev.calls, exact_hits=self.exact_hits, exact_tries=self.exact_tries)

    # ---- reading the result ----
    def principal_line(self, s, depth, target_index=0, max_turns=20):
        """Best plays from s, following the most likely outcome of each roll."""
        lines = []
        node = s
        d = depth
        while d > 0 and len(lines) < max_turns:
            if isinstance(node, ChanceNode):
                break
            if node.stage == 'over':
                break
            e = self.tt.get((node, d))
            if e is None or e.best is None:
                break
            p = e.best
            if len(self.targets) > 1 and (node.side == 'O' and node.stage == 'play'):
                p = max(e.children, key=lambda pv: pv[1][target_index].ev)[0]
            side = node.side
            nxt = apply_play(node, p)
            rolls = []
            while isinstance(nxt, ChanceNode):
                po, lab, sub = max(nxt.outcomes, key=lambda o: o[0])
                rolls.append((nxt.what, lab, float(po)))
                nxt = sub
            lines.append(dict(side=side, concept=p.concept, text=p.text(), rolls=rolls, play=p,
                              clock=node.clock, phase=node.phase))
            node = nxt
            d -= 1
        return lines, node
```

… lines 1262–1263: printed in the shared core (Part 1) …

```python
def describe_terminal(node):
    if not isinstance(node, ChanceNode) and node.stage == 'over':
        return str(score(node))
    return ''
```